

COMP2043.GRP Final Group Report

Image Classification/Recognition in a Federated Learning Framework (P2025-23)

Date: 1st April 2026

BY:

Aditya Agrawal (20617628, scyaa6)

MingYang Xue (20617494, scymx3)

XinChen Dai (20616310, scyxd8)

Sirui Li (20617511, scysl14)

Jiale Qin (20616174, scyj2)

Zhong Jun Khoo (20612661, hfyzk4)

Module name/number and Department

Software Engineering Group Project (COMP2043 UNNC)

Team name: Team.2025.18

Supervisor's name: Yanwen Mao

Word Count: 8390

Contents

1	Problem Description	1
2	Background & Research	3
2.1	Federated Learning for Image Classification	3
2.1.1	Image Classification	3
2.1.2	Federated Learning	3
2.2	Survey of Existing Systems	4
2.2.1	TensorFlow Federated (TFF)	4
2.2.2	FedML Framework	4
2.2.3	OpenFL Framework	4
2.3	Market Research	5
2.3.1	Industry Drivers	5
2.3.2	Market Gaps	5
2.3.3	Positioning of This Project	6
3	Requirements Specification	7
3.1	Functional Requirements	7
3.2	Non-Functional Requirements	8
3.3	Constraints and Assumptions	9
4	System Design	10
4.1	System Architecture Overview	10
4.2	Use-Case Diagram	10
4.3	Sequence Diagram	11
4.4	Activity Diagrams	12
4.4.1	Robustness Mechanism	12
4.4.2	Random Disconnect Handling	13
4.4.3	Privacy Mechanism	14
5	User Interface	15
5.1	System Overview	15
5.2	Server-Side User Interface	15
5.2.1	Server Dashboard	15
5.2.2	Training Control Panel	15
5.2.3	Training Mode Selection	16
5.2.4	Sample Training Mode	17
5.2.5	Logs and System Monitoring	17
5.2.6	Training Settings Panel	18

5.2.7	Data Display Panel	19
5.2.8	Graph Visualization Panel	19
5.3	Client-Side User Interface	20
5.3.1	Client Status Display	20
5.3.2	Dataset Label Selection	21
5.3.3	Client Setup Interface	21
5.3.4	Training Mode Selection	22
5.3.5	Update Transmission	23
6	Implementation Details	24
6.1	Prototype Using Flower (gRPC)	25
6.1.1	Sequence Diagram of Server–Client Interaction	26
6.2	Quadratic Model Demonstration	29
6.3	How the Training Workflow Works	29
6.4	Visualization and Logs	30
6.4.1	Activity Diagram Of a Federated Round	31
7	System Development and Prototyping	33
7.1	Aggregation Mechanisms: Parameter Aggregation	33
7.2	Data Processing Methods	33
7.2.1	Basic model	33
7.2.2	Dealing with random disconnect	34
7.2.3	Privacy Protection	36
7.2.4	Robustness	38
8	System Test Report	41
8.1	Testing Approach	41
8.1.1	Testing Strategy	41
8.1.2	Test Coverage	41
8.2	Test Environment	42
8.3	Test Cases and Results	43
8.3.1	Model Type Testing	43
8.3.2	Flower Communication Protocol Testing	44
8.3.3	Model Aggregation Testing	45
8.4	Key Test Results Details	45
8.4.1	FedAvg Aggregation Test (TC-A01)	45
8.4.2	Weighted Aggregation Test (TC-A04)	46
8.4.3	MAD+Reputation Defensive Aggregation (TC-A06)	46
8.4.4	Trust Update Test (TC-A07)	46
8.5	Test Summary	47

8.6	Test Execution Commands & Actual Execution	47
8.7	Test Conclusion	47
9	Problems Encountered	48
9.1	Technical Challenges	48
9.1.1	Unreliable Communication Between Server and Clients	48
9.1.2	Limited Training Visibility During Communication	48
9.1.3	Trade-off Between Security and System Performance	49
9.2	Teamwork Challenges	49
9.2.1	Language Barriers Among Team Members	49
9.2.2	Role Ambiguity and Task Overlap	50
9.2.3	Uneven Work Distribution	50
10	Summary and reflection	51
10.1	Summary and Reflection on the Project	51
10.1.1	System Performance	51
10.1.2	Communication Stability	51
10.1.3	Accuracy and Convergence Behavior	51
10.1.4	Comparison with Expected Results	52
10.1.5	Planned vs Actual Progress	52
10.1.6	Team Collaboration	52
10.1.7	Learning and Reflection (LSEPI)	53
10.2	Time Plan Table	53
11	Conclusion And Future Work	55
11.1	Summary of Achievements	55
11.2	Limitations	55
11.3	Future Improvements	56
11.3.1	More Improvements:	56
	References	57
	Appendices	60
A	Meeting Minutes	60
A.1	Meeting Minutes 1 (17 October 2025)	60
A.2	Meeting Minutes 2 (29 October 2025)	62
A.3	Meeting Minutes 3 (5 November 2025)	63
A.4	Meeting Minutes 4 (12 November 2025)	64
A.5	Meeting Minutes 5 (19 November 2025)	65
A.6	Meeting Minutes 6 (3 December 2025)	66

A.7	Meeting Minutes 7 (10 December 2025)	68
A.8	Meeting Minutes 8 (21 January 2026)	70
A.9	Meeting Minutes 9 (4 March 2026)	72
A.10	Meeting Minutes 10 (18 March 2026)	74
A.11	Meeting Minutes 11 (25 March 2026)	77
B	User Manual	79
C	Testing and Evaluation	85
C.1	Testing Overview	85
C.1.1	Testing Strategy	85
C.1.2	Test Coverage	85
C.2	Test Cases and Results	86
C.2.1	Model Type Testing	86
C.2.2	Flower Communication Testing	87
C.2.3	Model Aggregation Testing	88
C.3	Detailed Result Analysis	89
C.3.1	FedAvg Aggregation Test (TC-A01)	89
C.3.2	Weighted Aggregation Test (TC-A04)	89
C.3.3	MAD + Reputation Defensive Aggregation Test (TC-A06)	89
C.3.4	Trust Update Test (TC-A07)	90

1 Problem Description

There has been a dramatic increase in the quantity of very quality data available for doing training for all of these newly developed builders of large language models (“LLMs”). For example, when Meta released **Llama 3** in April 2024, it showed that it was significantly more advanced than **Llama 2** primarily because of the amount of high quality data that Meta had collected to build their very carefully curated training collections. As Meta stated, “Developing a large and high-quality training dataset through curation is a critical requirement” (Meta, 2024). Over the last few years the sentiment shared by Meta has continued to be replicated in the greater machine learning community; therefore, the higher quality and broader range of information that is built into training datasets will continue to result in better performing models.

It is becoming increasingly unreasonable to rely exclusively on publicly available datasets. Recent research indicates that, in the coming years, there is the potential for the public web to run dry of **high-quality textual data** by the mid-2030s, and for image data to be similarly diminished by 2060 (Hain, 2025). The enormous volume of private information that exists, such as patient information, logs from devices, legal/financial documentation, and information from sensors, will remain unavailable to us due to the legal/ethical/privacy constraints imposed upon them (URL: Educating Silicon, 2024). Most of this type of data is stored within decentralized locations (e.g., physicians’ offices, businesses, personal electronic devices, i.e. smartphones, laptops, cars, IOT devices, etc.) across a variety of locations.

Traditional machine learning assumes all training data can be centralized in one location, but this is impractical in many real-world scenarios. Sensitive personal information is governed by stringent regulatory frameworks, such as the EU’s GDPR (enforced in 2018), which imposes strict limits on data transfer and centralized storage (Wu et al., 2025). As a result, conventional centralized training is growing increasingly misaligned with modern privacy expectations and legal requirements.

Even classic distributed training fails to fully address the issue. While it distributes computational tasks across multiple machines, it usually requires **data sharing** or **frequent gradient exchanges**. These processes not only incur substantial communication costs but also risk unintended information leakages since gradients themselves can reveal sensitive details.

To solve the privacy nightmare of shipping everyone’s raw data to a central server, Google came up with **Federated Learning (FL)**. The basic idea is simple: devices work

together to train one global model while keeping their own data exactly where it is. The main building blocks of FL are:

- **No exchange of data:**

Only model updates or gradients travel over the network.

- **Secure update exchange:**

Those updates can be encrypted or deliberately noised to make it much harder for anyone to reverse-engineer private information.

- **Client robustness:**

Devices dropping offline, crawling along because of weak hardware, or even acting maliciously in some cases (Fang et al., 2024).

- **Scalability and flexibility:**

A new participant just connects to the server and starts contributing; no dataset upload required (Kairouz et al., 2019).

Even with these advantages, serious obstacles remain:

1. **System and communication heterogeneity:**

Huge differences in hardware and network quality across devices create synchronization bottlenecks and the well-known **straggler problem**.

2. **Statistical heterogeneity:**

Local datasets are typically **on-IID**, meaning each client’s data reflects its own unique usage patterns or environment (Mothukuri et al., 2021; Xie et al., 2024).

3. **Privacy and security risks:**

Clever attackers can sometimes extract sensitive information just from the gradients.

This project focuses on those weaknesses by designing a federated learning framework that prioritizes secure communication, smarter and customizable aggregation logic, and reliable coordination across a diverse fleet of client devices.

2 Background & Research

2.1 Federated Learning for Image Classification

Federated Learning (FL) is used in this work for image classification, as data is typically sensitive and distributed among institutions or devices. FL allows collaborative training and preserves the privacy of local data.

2.1.1 Image Classification

Image classification is the task of assigning labels to images according to their visual content. Contemporary methods are usually based on the use of deep CNNs that learn multi-level image representations and lead to a major increase in classification accuracy (Krizhevsky et al., 2012).

2.1.2 Federated Learning

FL is a distributed learning technique that enables multiple clients to jointly train a common global model without exposing raw data. Every client is able to train locally and then sends the model updates to a central server that aggregates these updates and redistributes the new global model in rounds (Yang et al., 2019). The horizontal FL architecture is depicted in Figure 1.

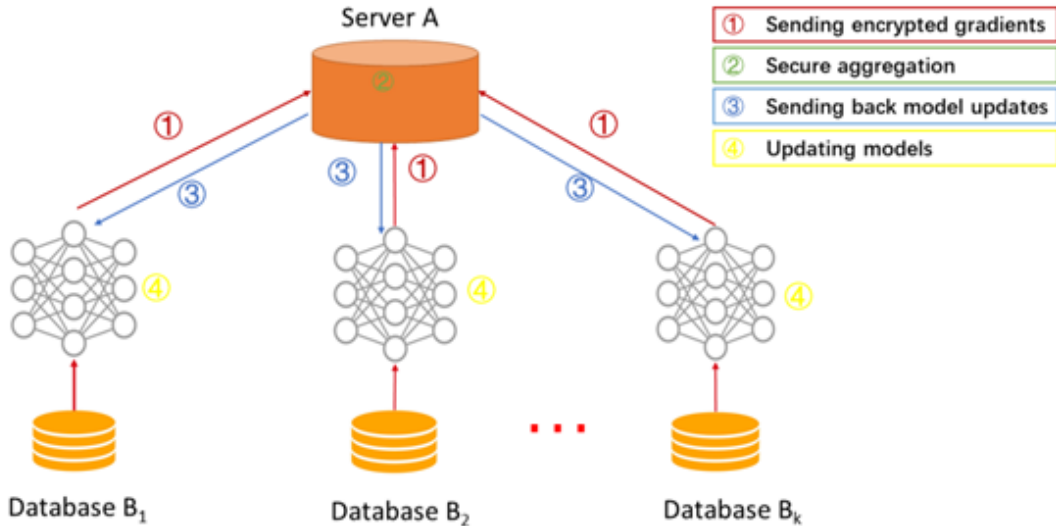


Figure 1: Architecture for a horizontal federated learning system

2.2 Survey of Existing Systems

In practice, a number of federated learning frameworks have been proposed in the last few years, concentrating on diverse aspects such as simulation, scalability, deployment, and privacy. This review discusses three examples of such systems and argues that their shortcomings provide an impetus for the design goals pursued in this work.

2.2.1 TensorFlow Federated (TFF)

TensorFlow Federated (TFF) is Google’s research-oriented FL system for decentralized optimization experiments, providing high-level APIs and simulation tools for rapid prototyping on datasets such as MNIST or CIFAR-10 (Reina et al., 2021).

However, TFF is designed not for real devices but for simulation, has no practical client-side administration tools, and does not have in-built monitoring (Ingerman & Shlens, 2020). These restrictions render it inapplicable for jobs that need closer integration with the devices, transparency, or flexible execution environments.

2.2.2 FedML Framework

FedML is a comprehensive FL system offering simulation environments, experiment tracking, benchmark datasets, and cross-device training, and is widely adopted due to its modular ecosystem (He et al., 2020).

Although powerful, FedML’s architecture is relatively heavy, with deeply abstracted internal operations that limit transparency and make fine-grained customization difficult (Kairouz et al., 2021). For small teams or prototype-focused projects, its complexity introduces unnecessary overhead in deployment and maintenance.

2.2.3 OpenFL Framework

OpenFL is designed for privacy-sensitive domains such as healthcare, enabling multi-institution model training without sharing raw patient data (Romanini et al., 2021).

However, it demands significant infrastructure resources and lacks GUI dashboards, real-time monitoring, and simple client management. Studies note that practical adoption requires more usable tooling (Sheller et al., 2020), making OpenFL unsuitable for flexible or interaction-oriented FL environments.

2.3 Market Research

Federated learning has gained massive traction thanks to the explosion of data living on edge devices and the increasingly strict privacy laws worldwide. Below is a breakdown of the main forces driving the industry, the gaps that still exist, and where this project fits in.

2.3.1 Industry Drivers

1. Privacy Regulations

Strict privacy laws such as the EU GDPR prohibit centralized storage of sensitive image data, including medical scans, biometric images, and camera footage (European Union, 2016). FL enables compliant distributed training by keeping such image data on-device and sharing only model updates.

2. Distributed Data Across Devices

Modern image data is naturally scattered across hospitals, smartphones, laptops, IoT cameras, and edge devices. Centralized ML cannot easily aggregate these fragmented visual datasets, whereas FL is explicitly designed to utilize such distributed image sources securely (Kairouz et al., 2021).

3. High-Value Industry Use Cases

- **Healthcare:** Medical imaging across institutions without exposing raw scans (Sheller et al., 2020).
- **Finance:** Document and ID-image verification with strict governance (Yang et al., 2019).
- **Autonomous Vehicles:** Large volumes of sensitive visual sensor data (Rieke et al., 2020).
- **IoT and Edge Networks:** Continuous image and video streams generated by edge cameras (Lim et al., 2020).

These image-centric industries collectively create strong market demand for FL systems capable of secure, distributed visual model training.

2.3.2 Market Gaps

Although FL adoption has expanded, several key gaps remain:

- Lack of accessible and intuitive user interfaces.
- Limited real-time monitoring and training visualization (He et al., 2020).

- Insufficient mechanisms for handling unstable or heterogeneous clients (Kairouz et al., 2021).
- Limited support for secure transmission mechanisms (Bonawitz et al., 2017).
- Limited flexibility in aggregation strategies.
- High deployment complexity in existing industrial systems (Romanini et al., 2021).

These limitations hinder widespread adoption, especially for scenarios requiring rapid setup or flexible experimentation.

2.3.3 Positioning of This Project

This project aims to address the above gaps by delivering:

- A lightweight FL system with minimal setup requirements
- Real-time dashboards for connectivity monitoring, learning curves, and logs
- Modular and customizable aggregation mechanisms
- Robust training under client disconnections and abnormal updates
- Encrypted model update transmission for enhanced privacy protection
- Suitability for prototyping and small-scale distributed experiments

By providing usability, clarity, and modularity, elements often lacking in existing FL systems (Kairouz et al., 2021). The proposed platform reduces the barriers to adopting federated learning.

3 Requirements Specification

This section describes the basic requirements that the federated learning system must meet. These requirements were developed in collaboration with the project team and supervisor and were refined according to the final implementation of the system.

3.1 Functional Requirements

The system shall be capable of the following:

1. **Client–Server Communication**

The system must be able to support the connection of multiple clients to the central server using the Flower framework.

2. **Model Distribution**

The server must be able to send the latest global model parameters to all the active clients at the start of each round of training.

3. **Local Model Training**

The system must be able to perform the local training process on each client’s private data, which includes forward propagation, loss computation, backward propagation, and updates.

4. **Transmission of Model Updates**

The system must be able to perform the local training process on each client’s private data, which includes forward propagation, loss computation, backward propagation, and updates.

5. **Model Structure Validation**

The server must validate model structures and parameter dimensions across clients and issue warnings in case of inconsistencies.

6. **Custom Aggregation Mechanisms**

The server should support aggregation of client updates through various mechanisms:

- Normal aggregation via FedAvg aggregation
- Weighted aggregation
- Defense-aware aggregation via MAD + reputation-based aggregation

7. **Privacy Protection Mechanism**

The system should support partial homomorphic encryption of model parameters via Paillier.

8. **Trust and Reputation Management**

The system should support trust scores for clients based on the quality of their updates.

9. **Global Model Redistribution**

The aggregated global model should be redistributed to all clients for the next round of training.

10. **Logging and Monitoring**

The system should support the following:

- Client connections and disconnections
- Training progress
- Aggregation
- Detection of anomalies

3.2 **Non-Functional Requirements**

The system should be able to meet the following quality and operational requirements:

1. **Privacy and Security**

The raw training data should not be transferred from the client device. The model updates should be secured using encryption and defensive aggregation techniques in order to prevent gradient leakage attacks.

2. **Scalability**

The system should be able to accommodate multiple active clients (around 3-10 clients) with consistent communication and aggregation performance.

3. **Portability**

The system should be able to run on multiple platforms (Windows, macOS, Linux) with standard Python environments.

4. **Communication Efficiency**

The system should be able to minimize communication bandwidth by only transmitting essential model parameters. Partial encryption should be supported instead of full encryption.

5. **Modularity**

The system architecture should be modular to allow modifications in the following components:

- Communication (Flower)

- Training logic
- Aggregation logic
- Privacy mechanisms

6. **Robustness**

The system should be able to withstand stability in the presence of:

- Client disconnections
- Network instabilities
- Heterogeneous client performance

7. **Transparency and Monitoring**

The system should allow real-time monitoring of training progress using appropriate levels of logging.

8. **Performance Efficiency**

The system should be able to complete training rounds within acceptable time constraints (around 10 rounds in a minute or less for small-scale deployment).

3.3 **Constraints and Assumptions**

1. **Flower as the Communication Layer**

The flower is used only for communication purposes, and other activities such as training, aggregation, and security are handled separately.

2. **Dataset Constraints**

The experiments are performed with relatively small datasets, such as MNIST, due to hardware constraints.

3. **Hardware Limitation**

The proposed system is designed to run on standard laptops without needing GPU acceleration.

4. **Partial Encryption Trade-off**

In order to strike a balance between security and performance, only some of the parameters are encrypted.

5. **Controlled Experimental Environment**

The security of the model is tested under controlled conditions with relatively simple datasets.

4 System Design

4.1 System Architecture Overview

The system is based on the client-server architecture with the Federated Learning paradigm. In this approach, the client undertakes the training process individually with the aid of the client's personal data.

The communication process is carried out with the aid of the Flower library. The library facilitates the communication process and the parallel training process. The system's architecture is modular-based and allows the independent modification of the communication process and the training process.

4.2 Use-Case Diagram

The use-case diagram illustrates interactions between the two main actors:

- **Server Operator:** Creates projects, configures settings, manages FL sessions, monitors training, and saves models.
- **Client User:** Connects to the server, loads datasets, accepts training requests, performs local training, and sends updates.

The diagram highlights the collaborative workflow and the fundamental FL principle that all raw data stays on client devices.

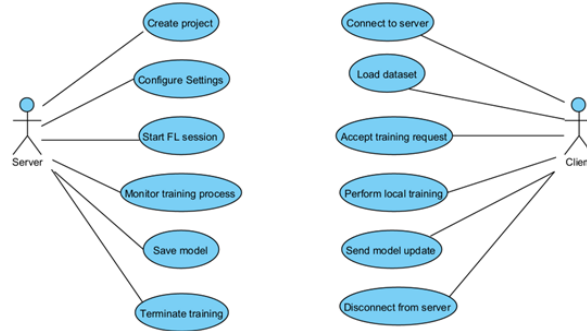


Figure 2: Use-Case Diagram of the Federated Learning System

4.3 Sequence Diagram

The sequence diagram illustrates the communication between clients and the server during training:

The server initializes the global model and shares it with the clients.

- **Client:** Receives the global model and updates it using their own data, sending the updates back to the server.
- **Server:** Collects the updates and combines them to form a new global model.

The Flower library facilitates communication and enables the execution of multiple clients in parallel.

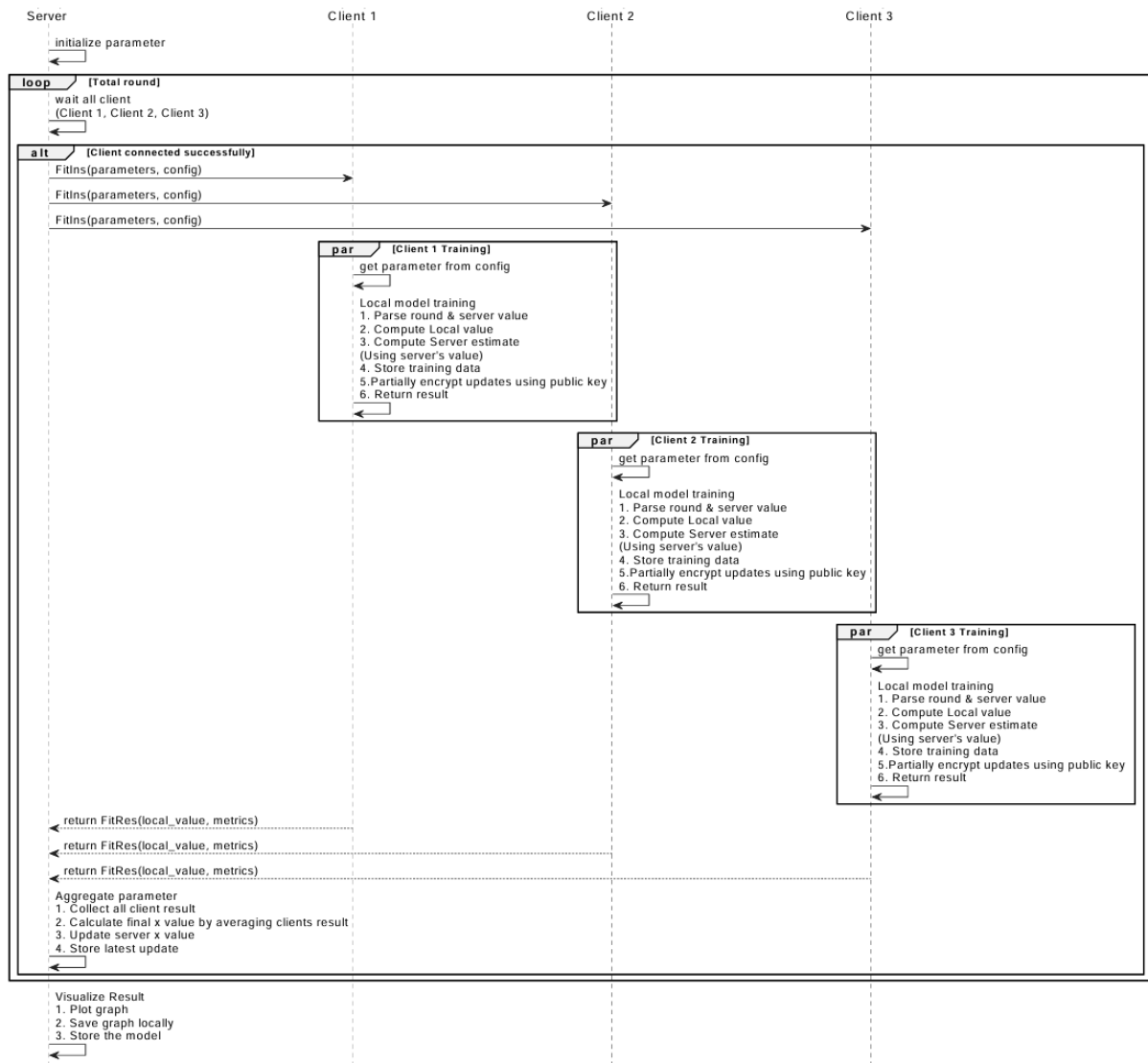


Figure 3: Sequence Diagram For Federated Learning System

4.4 Activity Diagrams

4.4.1 Robustness Mechanism

This diagram shows how the system deals with the unreliable behavior of clients:

- The server awaits model updates from all contributing clients.
- Clients that do not respond in time, according to the timeout value, are assumed inactive. The aggregation continues with the updates that are available.
- Anomalies in updates are recognized. Defensive aggregation is applied to minimize their impact on the global model.
- Dynamic updating of trust levels is used to reflect the reliability of contributing clients.

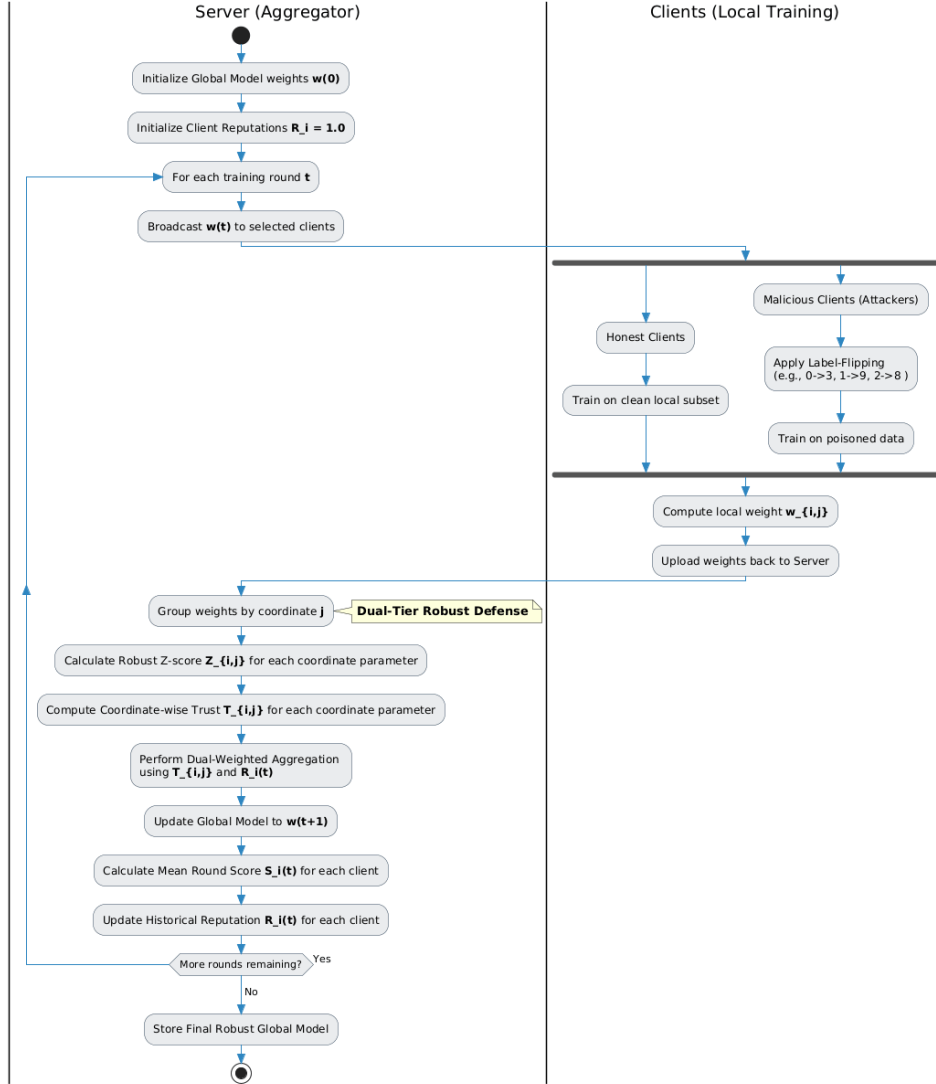


Figure 4: Robustness Activity Diagram

4.4.2 Random Disconnect Handling

This diagram represents situations where clients disconnect unexpectedly:

- The server recognizes disconnected clients using heartbeat messages or timeouts.
- Training is continued without waiting indefinitely for disconnected clients.
- Clients may rejoin at any time. The number of training rounds that were skipped is recorded.
- When clients reconnect, they obtain the latest global model. Training is continued from there.

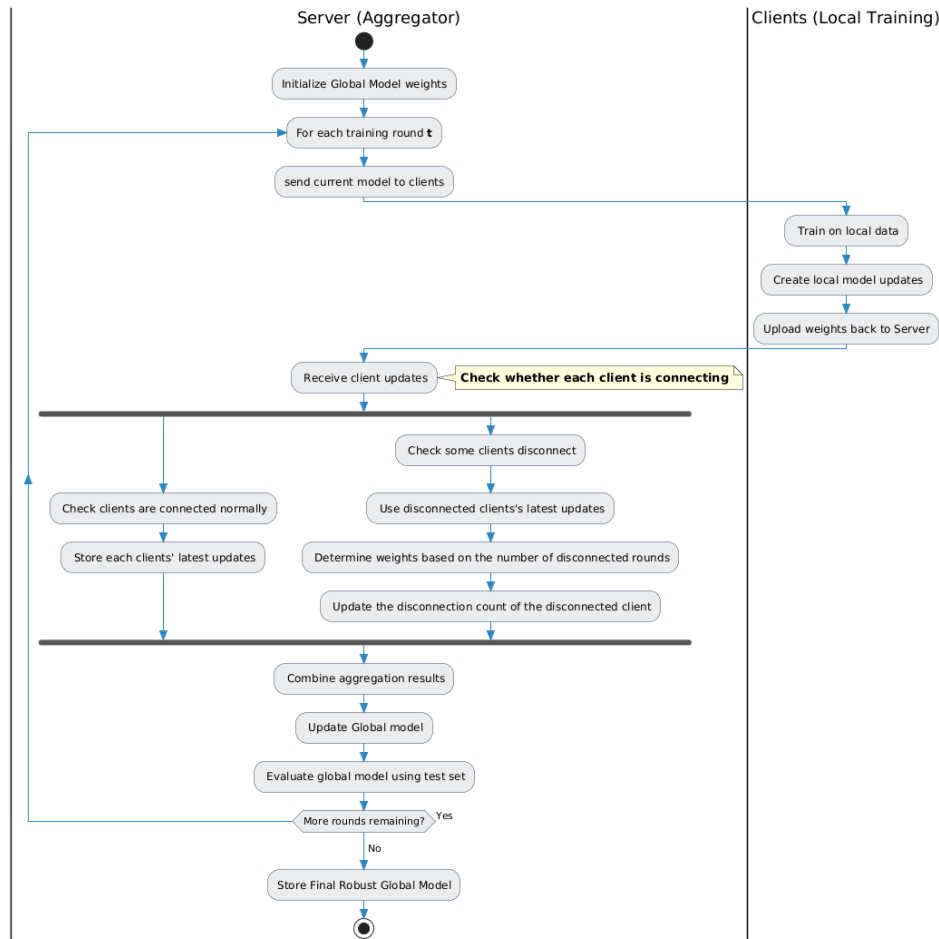


Figure 5: Random Disconnect Handling Activity Diagram

4.4.3 Privacy Mechanism

This diagram shows how the system ensures data privacy for clients:

- Clients obtain the global model from the server.
- Clients send updated model parameters
- Uploaded parameters are partially protected with homomorphic encryption

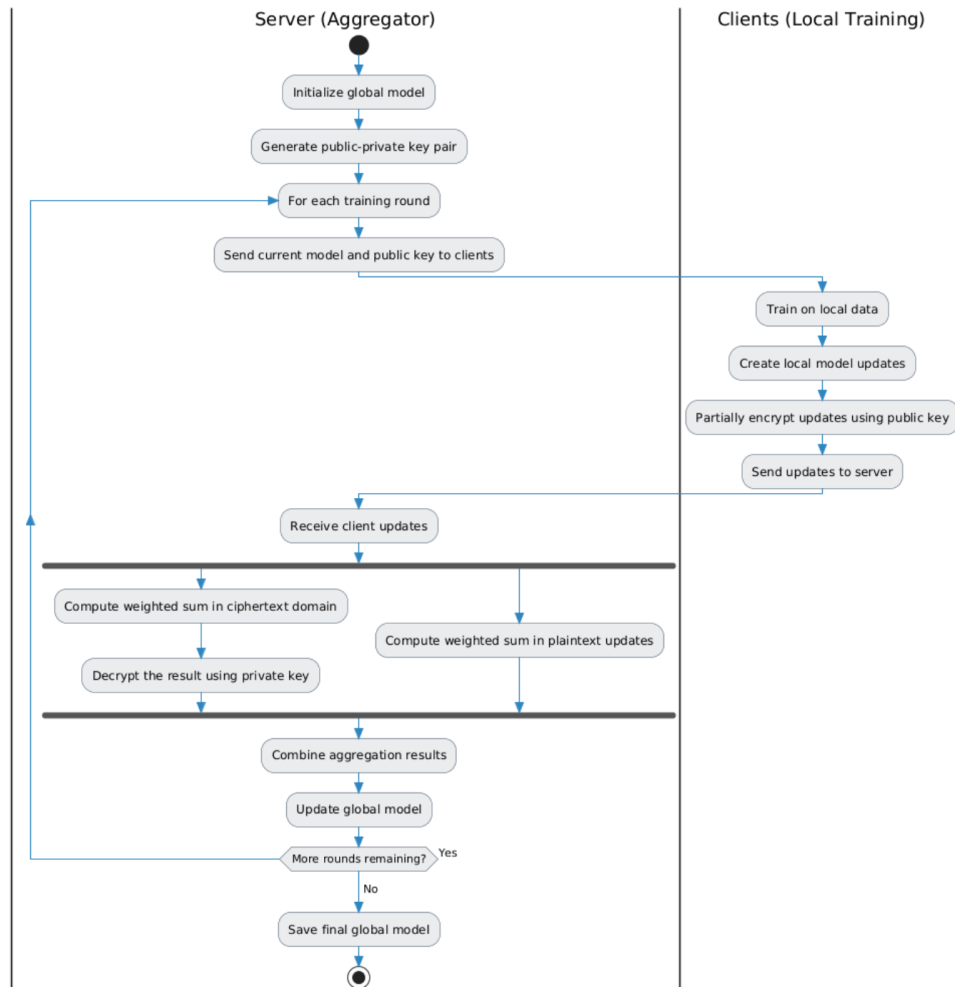


Figure 6: Privacy Activity Diagram

5 User Interface

5.1 System Overview

The system offers graphical user interfaces for server as well as client modes to make interaction with the federated learning framework easier. The graphical user interfaces are intuitive, ensuring that users can easily keep track of the system's behavior.

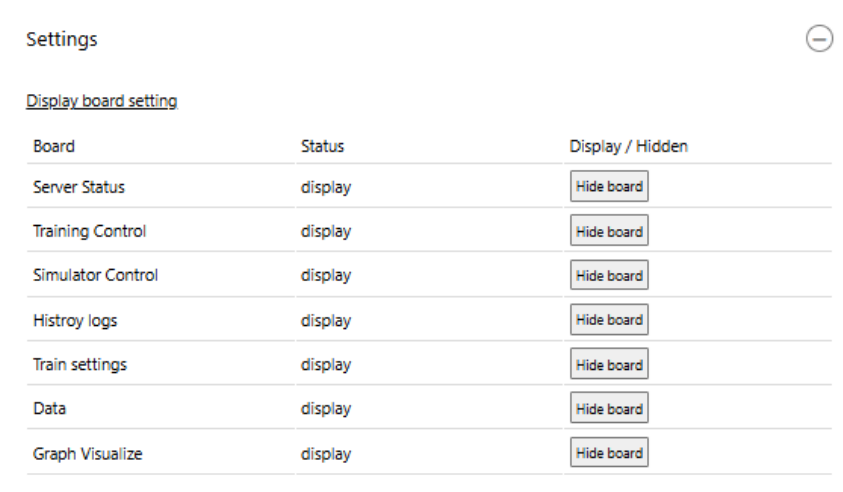
Both the graphical user interfaces support real-time updates, ensuring that the user is able to keep track of the training process as well as the communication process.

5.2 Server-Side User Interface

5.2.1 Server Dashboard

Once the server is launched, the system will present the main dashboard interface. The main dashboard interface will have a **Display Board Settings** panel that will allow the user to control the visibility of different components.

The components of the main dashboard interface, such as training control, logs, simulator, and dataset panel, may be controlled by the use of '**Hide**' and '**Display**' buttons. This is especially important for customizing the layout of the main dashboard interface for a specific purpose or task.



Settings		
Display board setting		
Board	Status	Display / Hidden
Server Status	display	Hide board
Training Control	display	Hide board
Simulator Control	display	Hide board
Histroy logs	display	Hide board
Train settings	display	Hide board
Data	display	Hide board
Graph Visualize	display	Hide board

Figure 7: Server Dashboard – Display Board Settings

5.2.2 Training Control Panel

The training control panel is the main point of interaction with the federated learning process.

Through this panel, the user is able to:

- Select the training mode
- Start the training process
- Monitor the current training rounds
- Terminate the execution if needed

After the selection of the mode and the start of the training process, the server will automatically control all the connected clients.

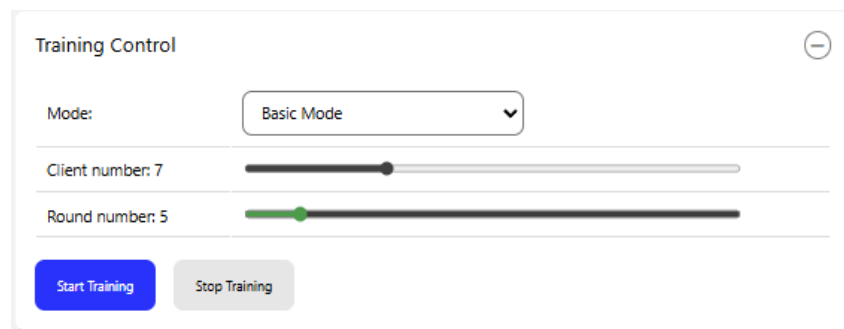


Figure 8: Server Training Control Panel

5.2.3 Training Mode Selection

The server also offers a variety of training modes that are accessed via a dropdown menu. The training modes enable the user to simulate different scenarios of federated learning. The training modes are:

- Basic Mode
- Availability Mode
- Homomorphic Encryption
- Robustness Attack No Defense
- Robustness Attack with Defense

These training modes modify the operation of the system. The robustness training modes, for instance, introduce malicious clients into the network, and the homomorphic encryption mode encrypts the updates before sending them.

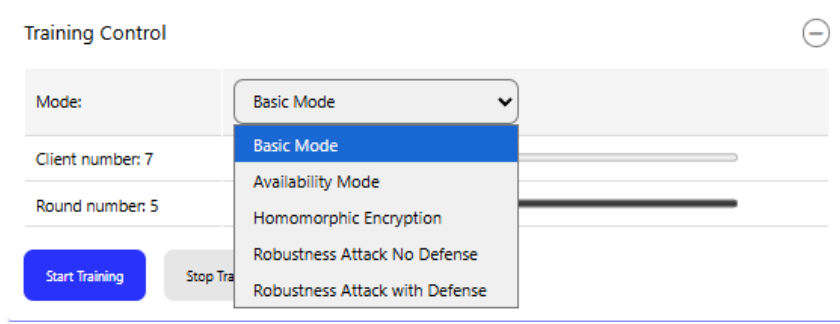


Figure 9: Server Training Mode Selection

5.2.4 Sample Training Mode

In addition to these modes, the server also includes a **Sample Training mode** for testing and demonstration purposes.

The module includes extended scenarios such as:

- Random Disconnect
- Robustness No Attack No Defense
- Robustness Comparison

These modes allow users to see the behavior of the system in controlled experimental conditions.

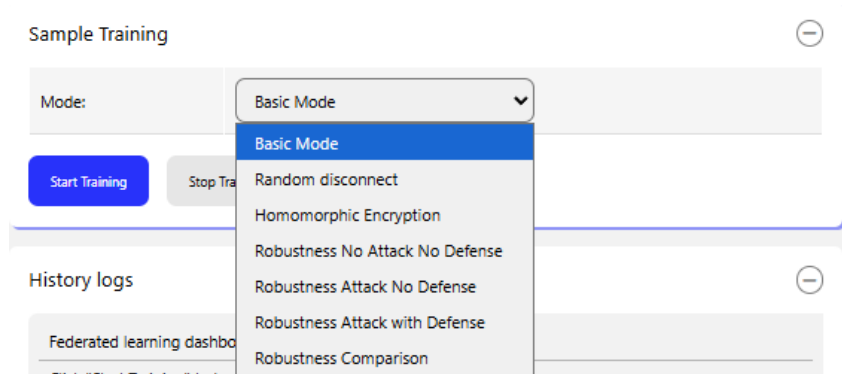


Figure 10: Sample Training Modes

5.2.5 Logs and System Monitoring

The server interface comprises a **message display panel**. It displays the real-time logs. This panel tracks the following activities:

- Client connections and disconnections
- Training progress updates

- Aggregation events
- Error messages

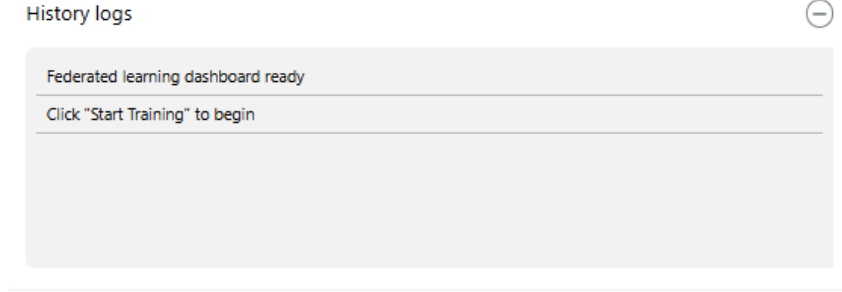


Figure 11: Server Logs Display Panel

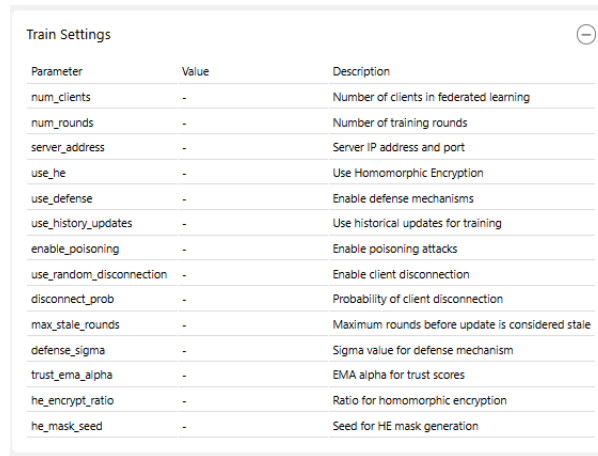
5.2.6 Training Settings Panel

The server interface also includes a Training Settings panel that allows the user to set some key parameters before the federated learning process is initiated. The Training Settings panel is a structured table where each parameter is mapped to a particular function in the system.

The key parameters include:

- **num_clients**: number of clients involved in the process
- **num_rounds**: number of training rounds to be carried out
- **server_address**: server address and port number for communication
- **use_he**: homomorphic encryption use flag
- **use_defense**: flag for activating defense mechanisms
- **enable_poisoning**: flag for activating poisoning attacks
- **use_random_disconnection**: random client disconnections
- **disconnect_prob**: client disconnection probability
- **defense_sigma** and **trust_ema_alpha**: parameters for defense mechanisms
- **he_encrypt_ratio** and **he_mask_seed**: homomorphic encryption parameters

The Training Settings panel provides a high level of flexibility for experimentation by allowing the user to simulate different conditions for federated learning.



Parameter	Value	Description
num_clients	-	Number of clients in federated learning
num_rounds	-	Number of training rounds
server_address	-	Server IP address and port
use_he	-	Use Homomorphic Encryption
use_defense	-	Enable defense mechanisms
use_history_updates	-	Use historical updates for training
enable_poisoning	-	Enable poisoning attacks
use_random_disconnection	-	Enable client disconnection
disconnect_prob	-	Probability of client disconnection
max_stale_rounds	-	Maximum rounds before update is considered stale
defense_sigma	-	Sigma value for defense mechanism
trust_ema_alpha	-	EMA alpha for trust scores
he_encrypt_ratio	-	Ratio for homomorphic encryption
he_mask_seed	-	Seed for HE mask generation

Figure 12: Training Settings Panel with configurable parameters

5.2.7 Data Display Panel

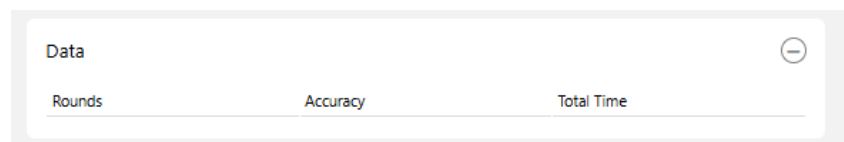
The server interface also includes a **Data panel** for showing key performance metrics during training:

The data is generally shown in a tabular form and includes:

- **Rounds:** The current round of training
- **Accuracy:** The accuracy of the model after each round of training
- **Total Time:** The total time consumed for training

The data is updated dynamically, and the user can monitor the improvement in the performance of the model over time.

This section is especially useful for checking the convergence of the training process and for comparison of different training modes.



Data		
Rounds	Accuracy	Total Time

Figure 13: Data Panel showing rounds, accuracy, and time

5.2.8 Graph Visualization Panel

In addition to the data table, the system also includes a **Graph Visualization panel** that will give a graphical view of training performance.

The graph will generally include:

- **Accuracy vs Training Rounds**

As training continues, the graph will update in real time. This will enable the user to easily see trends such as:

- Convergence speed
- Stability of performance
- Influence of different training modes

Graphical representation will also aid in easier interpretation of data compared to numerical data alone and will help in easily spotting problems such as a sharp drop in accuracy due to an attack or disconnection.

Example of a Graph:

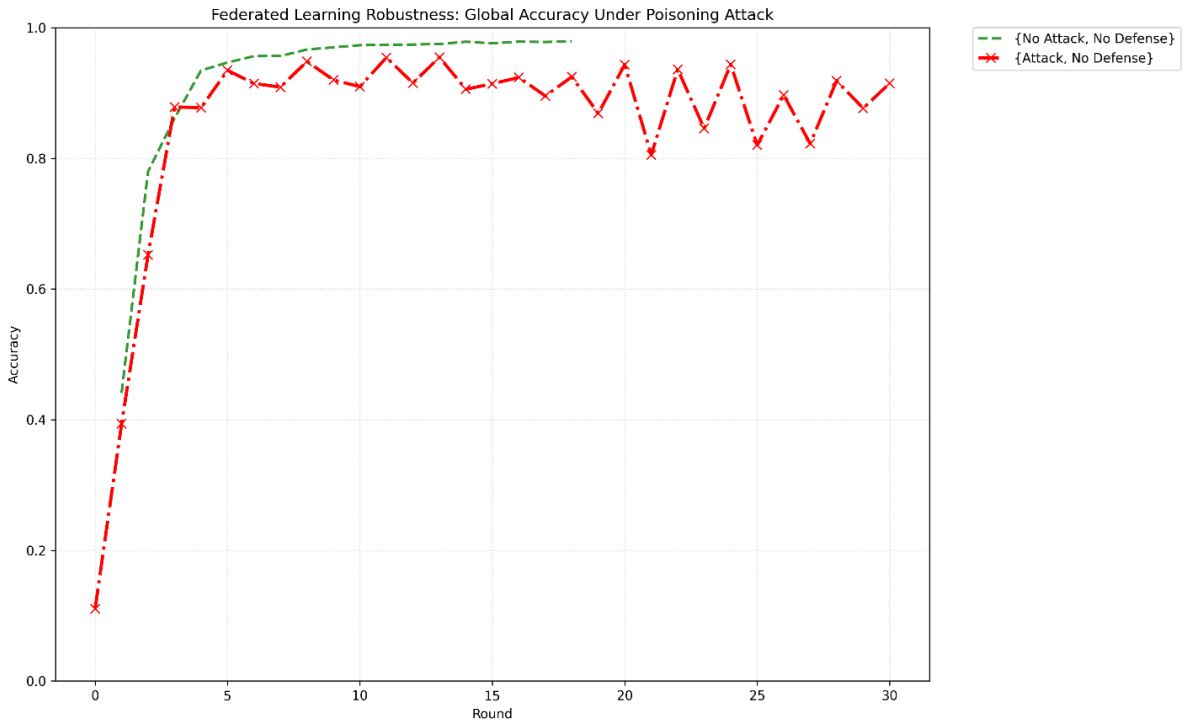


Figure 14: Accuracy vs Rounds Graph Visualization

5.3 Client-Side User Interface

5.3.1 Client Status Display

At the top of the client interface, there is a status bar that displays information about the current state of the client.

This includes:

- Current state of training (e.g., idle, training)
- Current round of training

This information lets users know whether the client is actively participating in a training session or is idle and awaiting instructions.



Figure 15: Client Status Bar

5.3.2 Dataset Label Selection

Below the status bar, there is a label selection panel. This feature enables a user to select particular labels (or classes) from the dataset for local training.

By selecting different labels on different clients, this system can mimic a situation where there is a non-IID data distribution, which is a realistic situation in FL. Once labels are selected, a client will train on a particular subset of data.

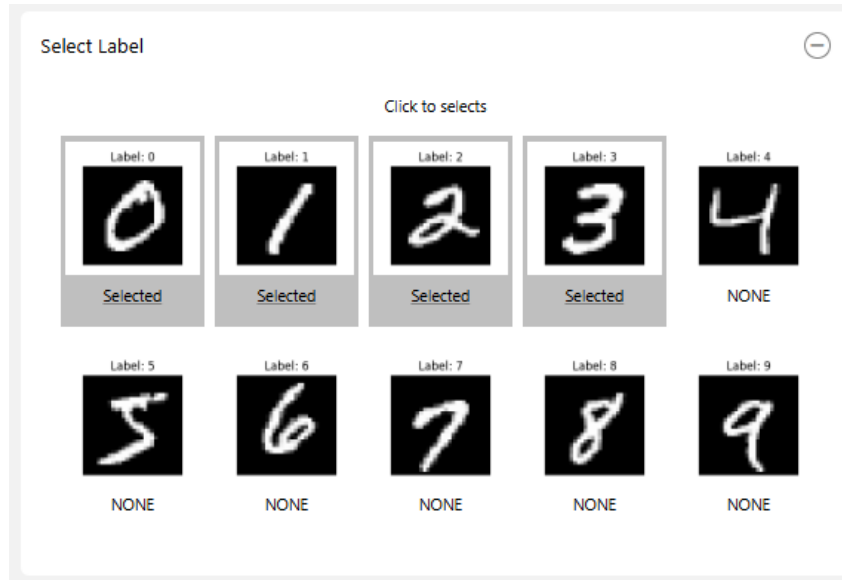


Figure 16: Dataset Label Selection Panel

5.3.3 Client Setup Interface

Once the client application is started, a configuration interface is presented to the user. The user can:

- Choose a Client ID (CID 1 – CID 8)

- Enter/Confirm Server Address

Once this is done, the client will connect to the server and await further instruction.

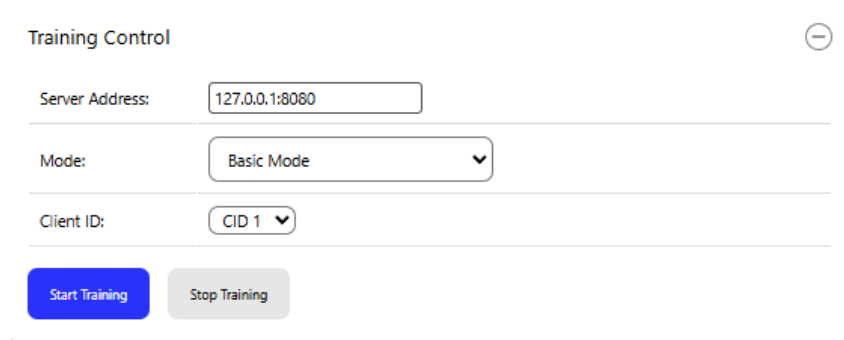


Figure 17: Client Setup Interface

5.3.4 Training Mode Selection

The client interface has a dropdown menu for choosing the mode of training. The modes of training include:

- Basic Mode
- Availability Mode
- Homomorphic Encryption
- Robustness Attack No Defense
- Robustness Attack with Defense

The mode chosen must be consistent with the server mode. After choosing a mode, the client waits for the server to start training.

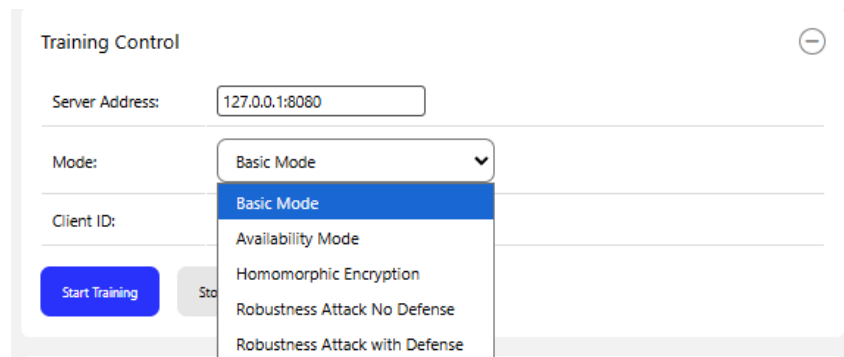


Figure 18: Client Training Mode Selection

5.3.5 Update Transmission

Once the local training is done, the client automatically sends the updated model parameters to the server.

This is reflected in the interface through status updates or a log showing that the client has finished its part for the current round.

No user interaction is required for this.

6 Implementation Details

This section outlines the main technical decisions behind the system, covering the front end, backend, communication framework, and theoretical foundations of the custom FL pipeline.

1. Frontend Implementation: HTML, CSS, JavaScript

These tools were used to create interactive components such as control panels, parameter settings, and real-time displays. JavaScript handles user actions (e.g., selecting modes, starting training) and updates the interface dynamically during execution.

2. Backend Programming Language: Python

We went with Python primarily because of its unmatched ecosystem for machine learning, its ability to run identically on Windows, macOS, and Linux, and how quickly it lets you turn ideas into working code. Out-of-the-box libraries took care of everything from tensor operations and model training to networking, detailed logging, and fast numerical work.

3. Work Hosting Platform: GitLab

We hosted the repository on GitLab, which served as our single hub for code versioning, task tracking, merge requests, and team collaboration throughout the project.

4. Communication Framework: Flower (gRPC-based)

The reason for introducing Flower was to enable communication between the client and the server. We did not solely rely on its built-in aggregation method; instead, we wrote our own corresponding logic to handle the features we added to the federated learning framework. Its reliance on gRPC allowed us to achieve fast, reliable, and bidirectional message exchange, while also providing excellent support for streaming large-scale model updates and gracefully handling timeout situations.

When deciding how to handle client–server communication, we weighed two primary paths: building a custom protocol on top of HTTP or using the existing Flower framework (which relies on gRPC).

A home-grown HTTP solution would have been straightforward to set up, universally compatible, and would have given us complete freedom over message structure, endpoints, and special features. On the flip side, we would have had to write from scratch a long list of low-level components—connection pooling, reliable message delivery, timeout handling, client reconnection logic, and proper synchronization across rounds. That alone would have added a huge amount of complexity and time to the project.

To explore the feasibility of the HTTP approach, we first built a simple prototype in Python that allowed a client to send JSON messages to the server and receive a response. This prototype demonstrated that HTTP can support basic message passing suitable for a minimal federated learning setup.

```
def functionPlusV(returnMessage, client_id, round_times):
    # Refactor variable
    originalInt = returnMessage['returnList']['client_int']
    incrementedInt = returnMessage['returnList']['return_int']

    # Find increment value (only for integer, float no doing)
    predictIncrement = incrementedInt - originalInt

    print(f"client:{client_id}, rounds:{round_times}")
    print(f"original value: {originalInt}, received value: {incrementedInt}")
    print(f"predict_increment: {predictIncrement}, server_generated_int: {returnMessage['returnList']['server_generated_int']}")
    correctness = (returnMessage['returnList']['server_generated_int'] == predictIncrement)
    print(f"validation ('correct' if correctness else 'false')\n")

    return {
        'client_id': client_id,
        'client_int': incrementedInt,
        'client_float': returnMessage['returnList']['return_float'],
        'content': 'Hello',
        'correctness': correctness,
        'roundtimes': round_times + 1
    }
```

Figure 19: HTTP-Based Client–Server Message Exchange Workflow

As shown in (Figure 19), the client sends an initial JSON request, the server processes it, performs a simple calculation, and returns a response. This confirmed that HTTP can function as a communication layer. However, when comparing this to our second option, using Flower we found that Flower’s gRPC-based communication significantly outperformed the custom HTTP approach in terms of speed, stability, and multi-client coordination. Because implementing an entire communication stack manually would require substantial effort, we chose not to continue with HTTP.

6.1 Prototype Using Flower (gRPC)

To validate Flower as the communication layer, we built a small test application. In each round of the demo:

- The server generated two random integers
- Broadcast them to all connected clients
- Clients computed their sum and returned it
- The server collected and displayed the results

Flower handled all message routing and client coordination, allowing us to focus on the computation rather than the underlying networking. The core client-side logic was implemented in the `fit()` function, which receives the server’s configuration message, performs the computation, and returns the result.

```

def fit(self, parameters, config):
    # Receive two numbers from the server and compute sum
    try:
        a, b, r = int(config["a"]), int(config["b"]), int(config["round"])
    except Exception as e:
        print(f"[{self.cid}] bad config: {config} ({e})")
        return parameters, 1, {"error": "bad_config"}

    s = a + b
    print(f"[{self.cid}] Round {r}: received a={a}, b={b} -> sum={s}")

    new_params = [np.array([0.0], dtype=np.float32)]

    metrics_to_send = {
        "sum": s,
        "client_id": self.cid
    }
    return new_params, 1, metrics_to_send

```

Figure 20: Client Fit Function for Arithmetic Test

On the server side, the key methods were `configure_fit`, which prepares the round configuration, and `aggregate_fit`, which gathers client responses and updates the global state.

```

# Dispatch random numbers (a, b) to all currently online client(s)
a, b = random.randint(1, 9), random.randint(1, 9)
print(f"[SERVER] Round {server_round}: send a={a}, b={b} to {len(clients)} client(s)")
ins = FitIns(parameters, {"a": a, "b": b, "round": server_round})
return [(c, ins) for c in clients]

```

```

def aggregate_fit(
    self, server_round: int, results: List[Tuple[ClientProxy, FitRes]], failures: List[BaseException]
):
    # If no client replies: log and return (None, {}), meaning no new global parameters this round
    if not results:
        print(f"[SERVER] Round {server_round}: no replies")
        return None, {}

    # If client replies: print message, get sum from metrics (client's 3rd return value).
    print(f"[SERVER] Round {server_round} results:")
    for cp, fr in results:
        s = fr.metrics.get("sum", None)
        client_id = fr.metrics.get("client_id", cp.cid)
        print(f" - from {client_id}: sum={s}")

    return results[0][1].parameters, {}

```

Figure 21: Server Aggregation and Message Handling

This experiment demonstrated that Flower could reliably manage multi-client communication and round-based message exchange, allowing us to proceed with a more meaningful federated learning demonstration.

6.1.1 Sequence Diagram of Server–Client Interaction

The sequence diagram illustrates how Flower coordinates communication during a federated learning round. It shows:

1. The server invokes `configure_fit` to broadcast round settings
2. Clients receiving the configuration and executing fit
3. Clients returning their results
4. The server calling `aggregate_fit` to compute new global parameters

This diagram clarifies how our custom aggregation logic integrates with Flower's workflow and visually represents the timing and order of interactions.

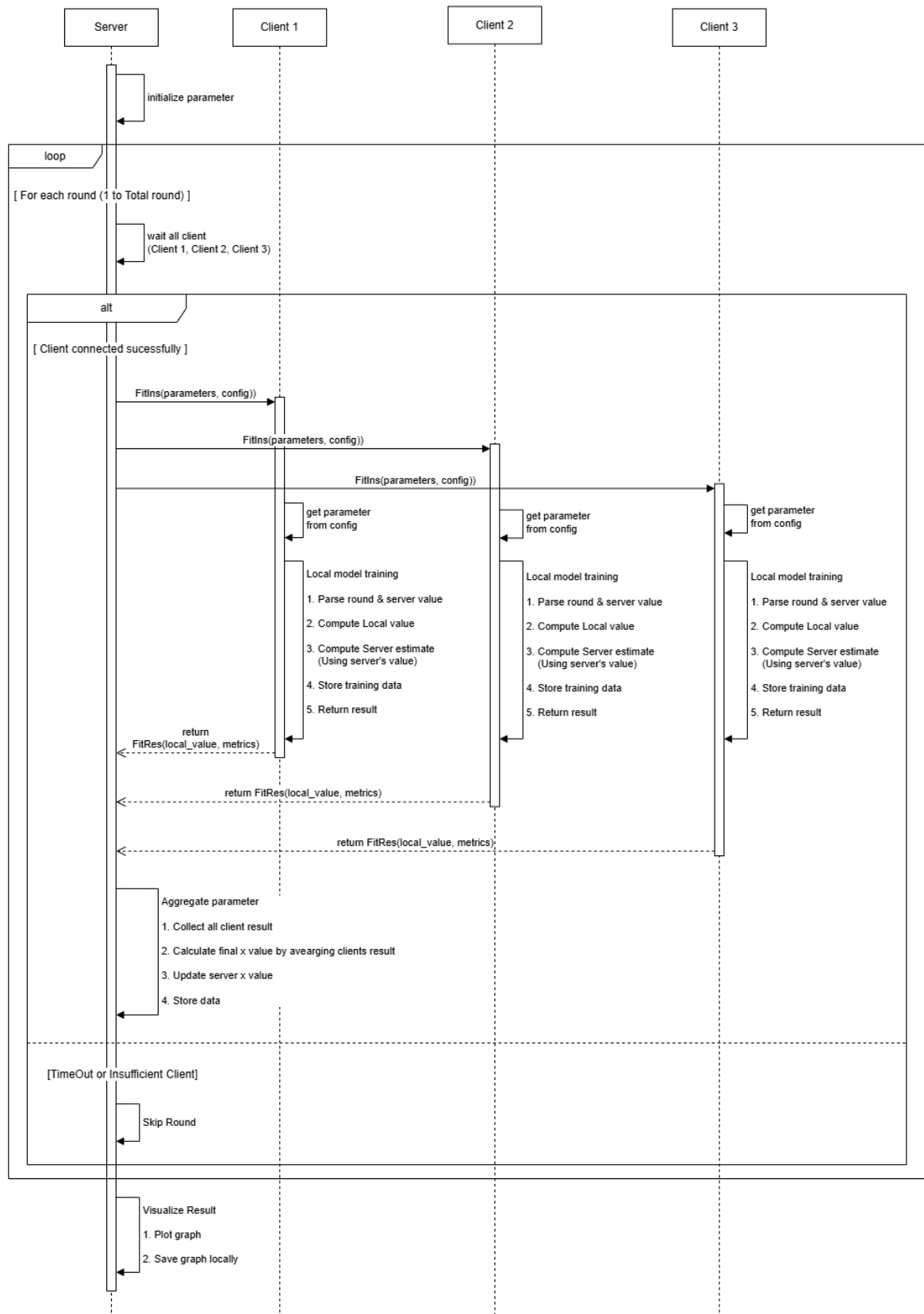


Figure 22: Server-Client Communication Sequence in Flower

6.2 Quadratic Model Demonstration

After validating Flower’s communication stability, we implemented a small federated optimization task using three quadratic functions—one assigned to each client. Clients received a shared parameter value from the server and computed local updates according to their individual objective functions. The server then aggregated these updates by computing their arithmetic mean.

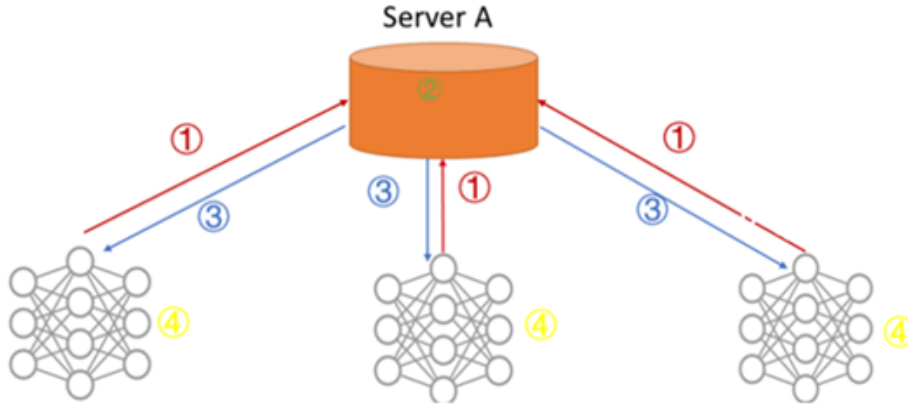


Figure 23: Overall Workflow for Quadratic Model Test

6.3 How the Training Workflow Works

The process follows these steps:

1. In Round 1, the server sends out an initial parameter value (`initial_x1`) to every connected client.
2. Each client takes that value, runs its own local update rule to calculate a new local parameter (`local_x1`), and also predicts what the next global value will be.
3. Clients send back their updated local parameter together with some metadata (their client ID and their predicted next global value).
4. The server waits for responses from the required number of clients. If not enough replies arrive before the timeout, the entire round is discarded, and the system moves on.
5. Once enough updates have been collected, the server combines the submitted `local_x1` values using the custom aggregation logic to produce the new official global parameter (`server_x_final`).

All parameters and metadata are saved for real-time visualization and to set up the configuration for the following round.

```

# record each client's data
for client_id, local_x1, client_server_x1 in client_data:
    #record client's local x1
    if client_id not in self.server_data["client_local_x1s"]:
        self.server_data["client_local_x1s"][client_id] = []
    self.server_data["client_local_x1s"][client_id].append(local_x1)

    # record client's calculated server_x1
    if client_id not in self.server_data["client_calculated_server_x1s"]:
        self.server_data["client_calculated_server_x1s"][client_id] = []
    self.server_data["client_calculated_server_x1s"][client_id].append(client_server_x1)

self.current_x1 = server_x_final # update server's x1
print(f"[SERVER] round: {server_round}: server calculate server_x_final={server_x_final:.4f}")
return ndarrays_to_parameters([np.array([server_x_final])]), {"server_x_final": server_x_final}
else:
    print(f"[SERVER] round: {server_round}: lack of client data, skip")
    return None, {}

```

Figure 24: Server Data-Collection Process (“Process 1”)

This test confirmed that Flower provides reliable round-based communication and that the system can support custom numerical update logic.

6.4 Visualization and Logs

To analyze system behavior, we implemented a plotting function that visualizes:

- Each client’s local parameter updates across rounds
- The server’s aggregated global parameter

This mirrors real federated learning behavior, where clients follow different trajectories due to data heterogeneity, and the server combines their updates into a single shared model.

Both server and client logs are included in the appendix.

Client logs record:

- Round Number
- Local update value
- Serve value Received

Flower also inserts “train message” identifiers for each communication round. After a client completes all rounds, it prints “Disconnected and shut down.”

Server logs include:

- Configure_fit – how each round is initialised,
- Aggregate_fit – number of responses collected and aggregated,

- `Configure_evaluation` – normally used for accuracy reporting (not applicable in this quadratic demo).

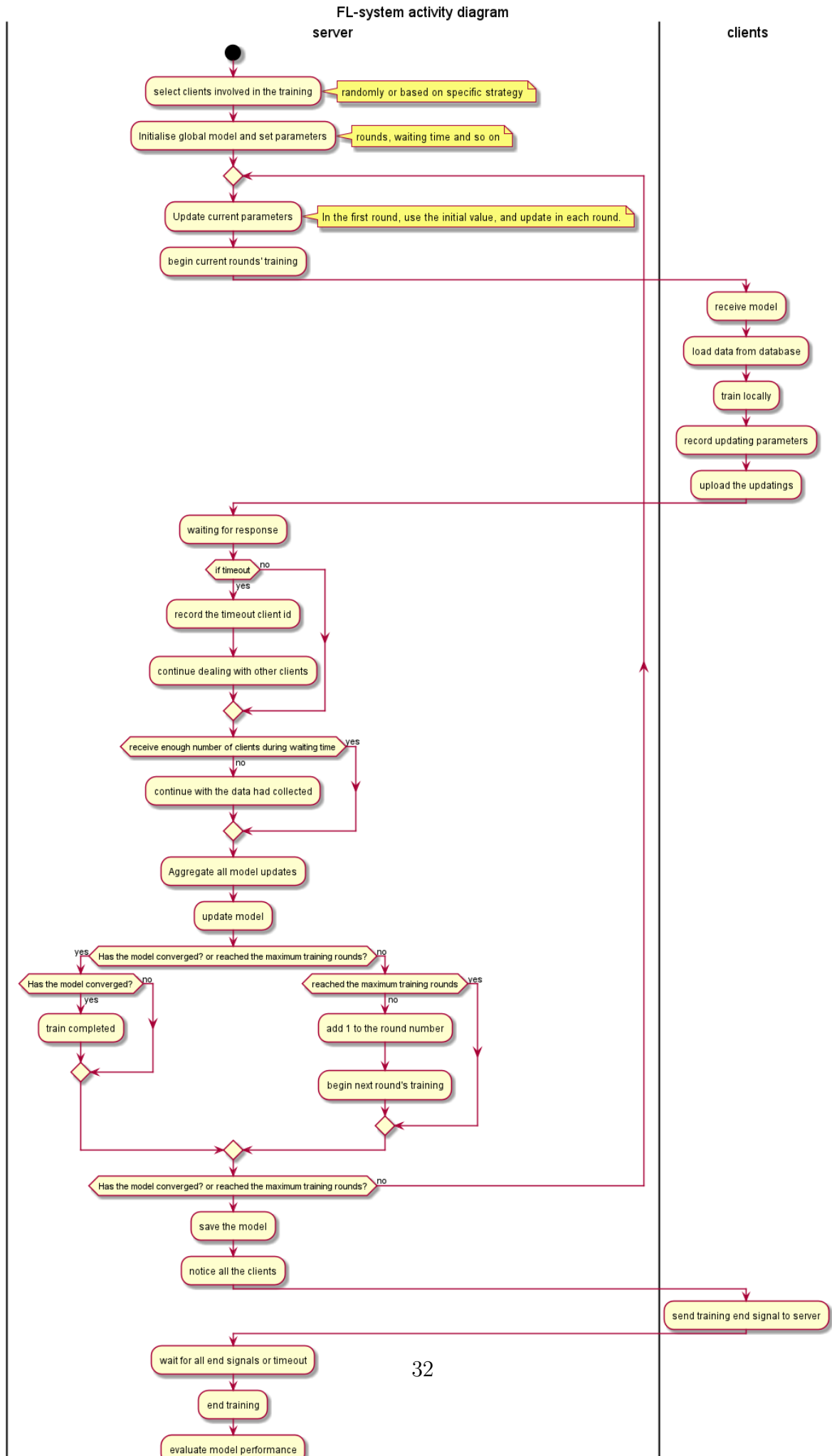
During 50 rounds of testing, no client disconnected, confirming that Flower ensures stable communication and is suitable for more complex federated learning tasks.

6.4.1 Activity Diagram Of a Federated Round

The activity diagram summarizes the full round workflow:

1. The server generates and broadcasts configuration parameters.
2. Clients compute local updates using their private data or local objective functions.
3. Clients send updated parameters to the server.
4. The server aggregates updates using the custom arithmetic-mean rule.
5. The resulting global model is stored and used to configure the next round.

This diagram highlights the iterative and decentralized nature of FL and shows how Flower supports synchronization without transferring raw data between devices.



7 System Development and Prototyping

7.1 Aggregation Mechanisms: Parameter Aggregation

We choose parameter aggregation as our aggregation method. The server waits synchronously for all three clients and computes the classical fedavg as global parameter. Without setting any additional weights and without weighting based on sample size, the fedavg can be considered as arithmetic mean. Follow the formula below:

$$x_{\text{global}}^{(t+1)} = \frac{1}{k} \sum_{i=1}^k x_i^{(t+1)}.$$

Below is a schematic diagram illustrating the data exchange between the server and the client when using parameter aggregation.

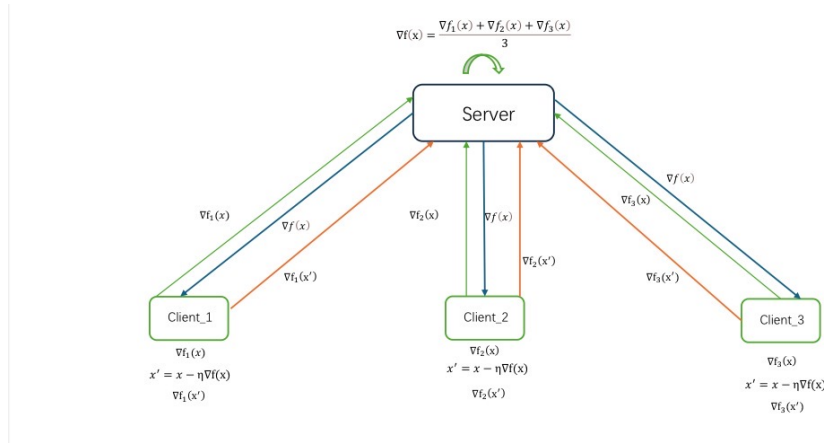


Figure 26: Parameter Aggregation

7.2 Data Processing Methods

7.2.1 Basic model

Data partitioning method:

We divide the data using the Non-IID method, where each client holds samples of different types of labels. By using this approach, we can demonstrate that federated learning performs well in the case of unevenly distributed data.

Server side:

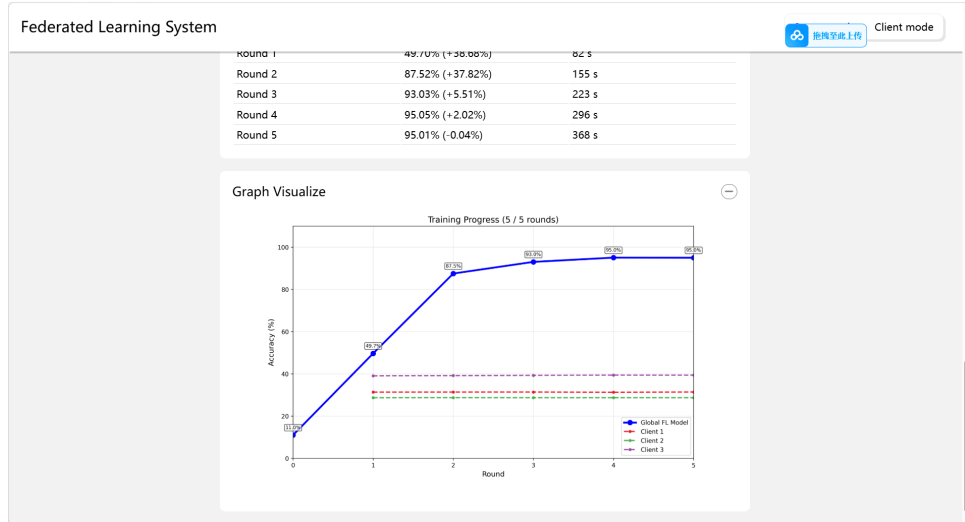


Figure 27: Server-side FedAvg aggregation and model distribution workflow

The server receives the model parameters uploaded by the client, calculates the global model using fedavg, evaluates the accuracy rate, and then distributes it to the client.

Client side:

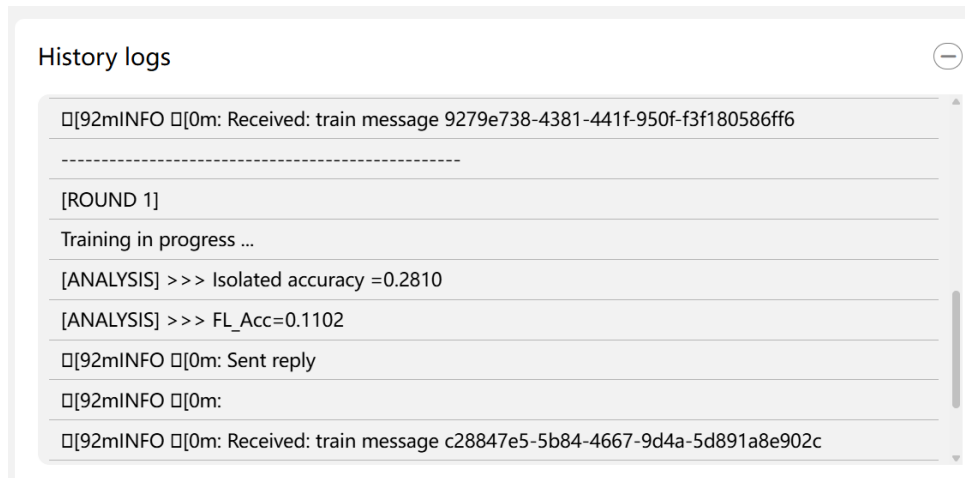


Figure 28: Client-side local training and model upload process

The client receives the global data distributed by the server, uses the local dataset for training, evaluates the accuracy of the local model, and then uploads the model to the server.

7.2.2 Dealing with random disconnect

In an ideal network environment, we built a federated learning framework with 3 clients, ensuring that the clients remained online during the training process. As the picture showed in last paragraph shows that the model can converge in less than 10 iterations,

and the model's global accuracy is above 95%.

But in the real life, people can not make sure the internet connection is stable. The connection situation is not only affected by client's own network environment, but also is restricted by server's bandwidth when conducting large-scale data training.

In order to simulate this situation, each client has a 20% chance of losing connection. The disconnection has affected the speed and effectiveness of the model training. It needs more rounds to converge. And the graph shows that each disconnection will lead to a drop of accuracy of the model. Although we can see the highest point of the accuracy graph is more than 90%, but this is not acceptable in actual industry practice. The model may not be in a relatively accurate state at the end of the training process.

To solve the above problems, we proposed an optimization strategy based on historical update storage and weight decay. The server stores the latest model update parameters of each client; when a client disconnects, the server calls its historical update parameters to participate in the global model average calculation. At the same time, considering that long-term disconnection will lead to the obsolescence of history parameters, that's why a weight decay mechanism is added. The longer the client is disconnected, the lower the weight of its historical updates as a substitute to participate in the current update, which can reduce the negative impact of outdated parameters on model training.

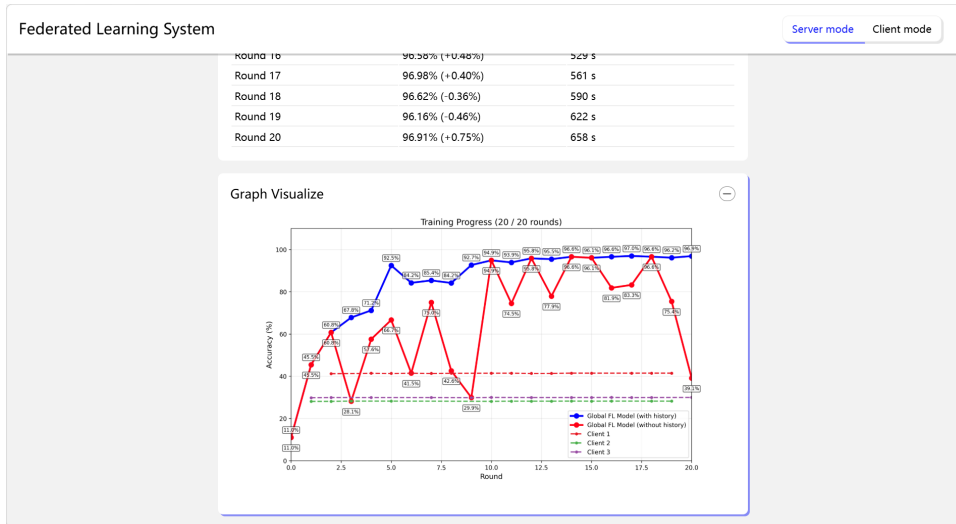


Figure 29: Training with random client disconnections using historical updates and weight decay

7.2.3 Privacy Protection

Although federated learning prevents direct data sharing, it remains susceptible to privacy leakage, since model updates have been shown to enable the reconstruction of sensitive training data (Zhu et al., 2019; Geiping et al., 2020). To mitigate this risk, this project implements a partial homomorphic encryption (HE) mechanism based on the Paillier cryptosystem (Paillier, 1999), which supports additive operations on encrypted data.

Specifically, the server first generates a Paillier key pair at initialization and distributes the public key to all clients, while retaining the private key locally. After local training, each client applies a deterministic mask, defined by a fixed random seed and encryption ratio, to select a consistent subset of model parameters for encryption across all participants. The selected parameters are quantized and encrypted using the public key. In the Paillier cryptosystem, a plaintext value m is encrypted as:

$$E(m) = g^m \cdot r^n \bmod n^2$$

where n and g are public key parameters, and r is a randomly chosen value. The remaining parameters are kept in plaintext, forming a mixed encrypted–plaintext representation of model updates.

During aggregation, the server computes a sample-size-weighted sum on mixed updates: encrypted entries are summed homomorphically in Paillier ciphertext, while plaintext entries are summed normally. It then decrypts only the aggregated encrypted entries, merges them with plaintext entries by index, and divides by total samples to obtain the new global model. This partial-encryption design stays compatible with standard federated learning and is much cheaper than full encryption.

The effectiveness of the proposed privacy mechanism is evaluated through an eavesdropping attack under a controlled experimental setting.

In our setup, a single client participates in federated learning using a simplified dataset (digit “0” only), as illustrated in Figure 1, and performs one-step local training per round. An external eavesdropper is deployed on the server side and intercepts both the global model parameters and the client-uploaded updates during each communication round. This setup follows an honest-but-curious threat model, where the training process remains unchanged, but the eavesdropper attempts to infer private data from the observed updates.

To perform such an inference, the attacker first computes the model update as:

$$\delta = \theta_u - \theta_g,$$

where θ_u denotes the uploaded model parameters, and θ_g denotes the global model parameters. In practice, δ consists of parameter tensors from different layers, obtained by subtracting global parameters from uploaded ones. These tensors describe how the model is adjusted during training.

From the attacker’s perspective, δ reveals how the model responds to an input and provides clues about the underlying data. The attacker starts with a random dummy input and iteratively refines it by matching its simulated update to the observed δ . As the difference decreases, the dummy input gradually approaches the reconstruction of the original training sample.

When encryption is disabled, the eavesdropper is able to reconstruct a recognizable input image, as illustrated in the second picture of (Figure 30), confirming that model updates leak sensitive information. In contrast, when the proposed partial homomorphic encryption mechanism is enabled, the intercepted updates become partially encrypted or obfuscated, preventing effective gradient matching. As a result, the reconstructed image degrades significantly and no longer reveals meaningful information about the original input, as shown in the third picture of (Figure 30).

These results demonstrate that the proposed mechanism effectively mitigates gradient-based privacy attacks under a realistic eavesdropping scenario, while preserving the standard federated learning workflow.

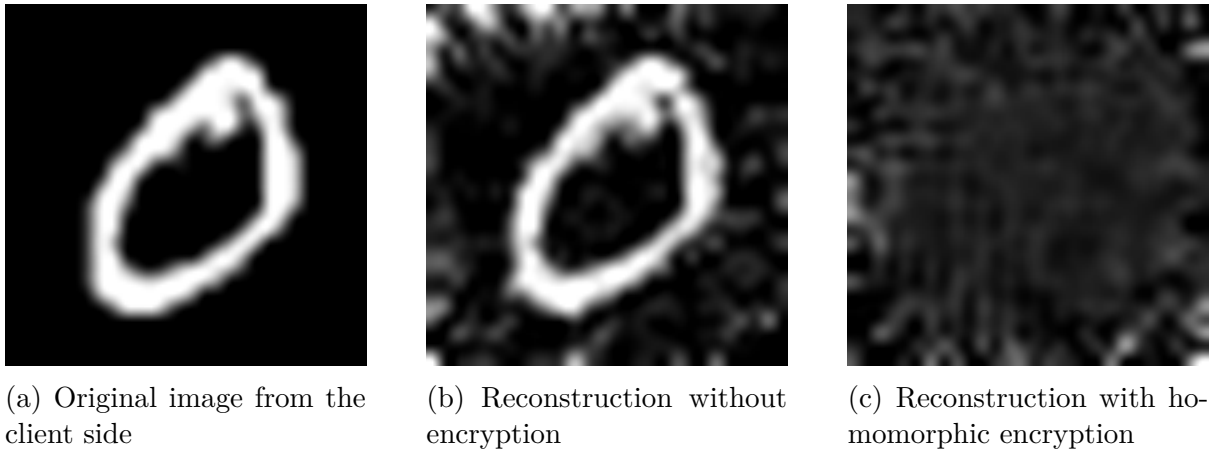


Figure 30: Privacy attack reconstruction results under an eavesdropping scenario.

7.2.4 Robustness

To simulate a realistic, highly skewed Federated Learning environment, we deploy the 8 clients in a Non-IID manner and a central server. An overlapping label-window strategy is employed, for instance, Client 1 exclusively contains images of digits [0,1,2,3], Client 2 holds [1,2,3,4], wrapping around to Client 8 with [8,9,0,1]. Consequently, each client possesses a severely biased local training subset of approximately 24000 images. Conversely, the global testing set of 10000 images remains unmodified and IID, serving as a unified benchmark to evaluate the generalization capability of the aggregated global model.

As an innovative distributed machine learning paradigm, Federated Learning (FL) has demonstrated significant application value in recent years within privacy-sensitive domains such as healthcare and financial risk control. However, its distributed nature also introduces severe security challenges. Among these, the Label Flipping Attack (LFA) has become one of the most threatening methods due to its high stealthiness and ease of implementation (Jia et al., 2026).

In our experimental setup, to simulate an attack, the training labels for Client 01 were tampered with while keeping its features intact. Our results show that using a simple weighted average aggregation causes a decline in global accuracy, accompanied by noticeable fluctuations. However, by incorporating specific defense mechanisms, experiments prove that the server can effectively resist Label Flipping Attacks (LFAs). As illustrated in the figure, the defensive performance is significant under the same experimental conditions.

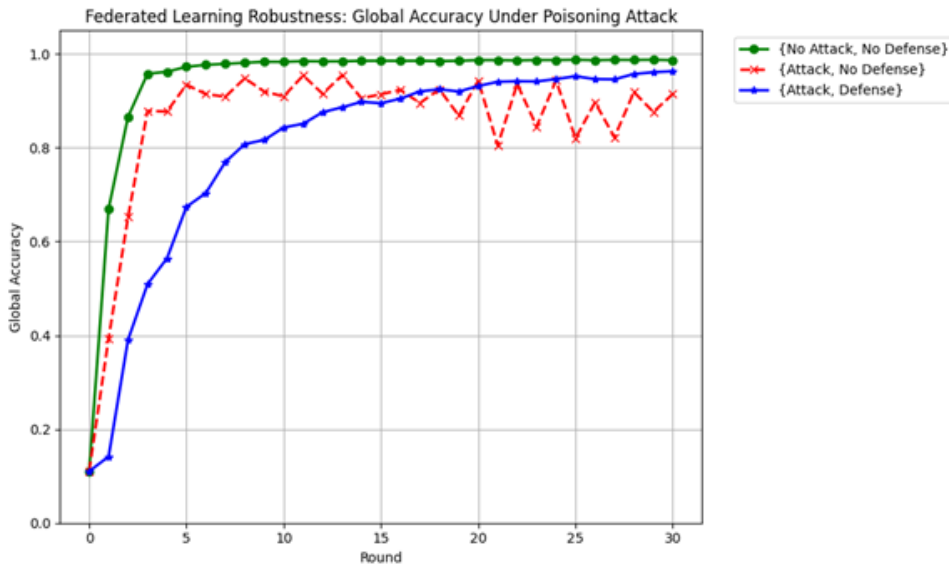


Figure 31: Robustness global accuracy comparison in three different scenarios

The server uses a Two-Tier Robust Aggregation Mechanism. It not only defends in terms of the parameters sent by the clients, but also actively tracks client behavior over time using a historical reputation system.

7.2.4.1 Coordinate-wise MAD Trust Evaluation

- **Compute the Deviation MAD**

For each parameter in each layer of the network, the server first computes the median of that parameter across all clients. It then calculates the absolute deviation of each client from this median and finally computes the median of these absolute deviations (MAD).

- **Robust Z-score $Z_{i,j}$**

This project uses Robust Z-score calculation method based on MAD to measure the abnormality of a client's parameter (Iglewicz & Hoaglin, 1993).

- **Calculate the coordinate trust $T_{i,j}$**

Those Z-scores are turned into a trust weight between 0 and 1 with a Gaussian weighting formula.

This project uses a Gaussian weighting formula to convert these Z scores into a trust weight between 0 and 1

$$Z_{i,j} = \frac{0.6745 \cdot |w_{i,j} - \text{median}_k(w_{k,j})|}{\text{MAD}}$$

$$T_{i,j} = \exp\left(-\frac{Z_{i,j}^2}{2\sigma^2}\right)$$

Definition of indices:

- i denotes the index of a specific client participating in the Federated Learning system.
- j denotes the position among the weight parameters within the neural network model.

7.2.4.2 Historical Reputation System

- **Round Trust Score S_i^t**

At the end of each communication round, every client is evaluated and assigned a new round score, denoted as S_i .

- **Reputation Update Score R_i^t**

At the beginning of the training process, the initial trust value for every participating client is identically set to 1.0. A client global reputation is determined by a weighted combination of its historical reputation and its current round score, controlled by α . This means that a client with a long history of good behavior, if suddenly compromised to perform poisoning attacks, will see its reputation drop rapidly; conversely, a client with a history of attacks will require a long time to rebuild trust.

$$S_i^t = \frac{\sum_j T_{i,j}^t}{|W|}$$

$$R_i^{(t)} = \alpha R_i^{(t-1)} + (1 - \alpha) S_i^{(t)}$$

- t denotes the current round
- $|W|$ denotes the total number of parameters in the neural network model

7.2.4.3 Final parameter aggregation

The final global model parameter $w_j^{(t+1)}$ is aggregated by combining both the coordinate trust of current round and the historical reputation:

$$w_j^{(t+1)} = \frac{\sum_i (T_{i,j}^{(t)} \cdot R_i^{(t)}) \cdot w_{i,j}^{(t)}}{\sum_i T_{i,j}^{(t)} \cdot R_i^{(t)}}$$

8 System Test Report

8.1 Testing Approach

This project adopts a layered testing strategy to ensure the functional correctness of the federated learning system, the reliability of the communication protocol, and the accuracy of the aggregation algorithm. Testing covers three core modules: model type detection, Flower communication protocol, and model aggregation functionality.

8.1.1 Testing Strategy

Test Type	Test Object	Test Framework	Coverage Content
Unit Testing	Independent module functions	Python unittest	Model structure detection, parameter shape validation, aggregation calculation, trust score updates
Integration Testing	Client-server interaction	Flower + unittest	Handshake protocol, heartbeat mechanism, parallel execution, timeout handling, reconnection mechanism
Defense Mechanism Testing	Anomaly detection and aggregation	Custom + unittest	MAD+Reputation defensive aggregation, outlier filtering, dynamic trust score updates

8.1.2 Test Coverage

- **Model Type Detection Module:** Verifies that the system can identify different model structures and trigger warnings.
- **Flower Communication Protocol Module:** Verifies the correctness and robustness of client-server communication sequences.
- **Model Aggregation Module:** Verifies the mathematical correctness of FedAvg aggregation, weighted aggregation, and defensive aggregation.

8.2 Test Environment

Item	Specification
Operating System	macOS
Python Version	3.10
Test Date	2026-03-22
Test Framework	Python unittest
Federated Learning Framework	Flower
Machine Learning Library	PyTorch
Numerical Computing Library	NumPy
Test Execution Environment	Anaconda3 (FAIens virtual environment)

8.3 Test Cases and Results

8.3.1 Model Type Testing

ID	Test Case	Description	Status
TC-M01	test_model __structure_detection	Verify system can detect differences in model structures	Pass
TC-M02	test_server __parameter_comparison	Verify server can detect parameter count differences	Pass
TC-M03	test_warning __trigger_condition	Verify warning triggers when parameters mismatch	Pass
TC-M04	test_client __parameter_shape_logging	Verify server records parameter shapes for each client	Pass

Table 1: Model Type Testing Results (4 tests passed, duration 0.005s)

Key Test Details:

- Successfully identified structural differences between standard and custom model layer configurations.
- Accurately detected parameter count discrepancies (10 vs 8 parameters).
- Triggered precise warnings for parameter shape mismatch with client ID and detail information.
- Correctly logged parameter shapes of multiple clients in standard format.

8.3.2 Flower Communication Protocol Testing

ID	Test Case	Description	Status
TC-F01	test_client __handshake_sequence	Verify client handshake sequence correctness	Pass
TC-F02	test_heartbeat __protocol	Verify heartbeat communication works normally	Pass
TC-F03	test_parallel __client_execution	Verify multiple clients execute in parallel	Pass
TC-F04	test_round __completion_sequence	Verify complete training round sequence	Pass
TC-F05	test_client __timeout_detection	Verify timeout client detection	Pass
TC-F06	test_reconnection __handling	Verify disconnected client reconnection	Pass

Table 2: Flower Communication Protocol Testing Results (6 tests passed, duration 0.417s)

Key Test Details:

- Handshake sequence strictly matched the expected connect → acknowledge → get_parameters → parameters.
- Heartbeat protocol generated stable time-stamped heartbeat signals.
- 4 clients achieved parallel execution with consistent start/end time stamps.
- Complete training round sequence executed in standard order, aggregation triggered only after all client updates.
- Accurately detected 1 timed-out client in test round.
- Successfully recorded and handled client reconnection with missed round statistics.

8.3.3 Model Aggregation Testing

ID	Test Case	Description	Status
TC-A01	test_fedavg __aggregation	Verify standard FedAvg aggregation	Pass
TC-A02	test_single __client_aggregation	Verify single client aggregation	Pass
TC-A03	test_empty __client_handling	Verify handling when no clients participate	Pass
TC-A04	test_weighted __aggregation	Verify weighted aggregation based on sample sizes	Pass
TC-A05	test_he __aggregation_structure	Verify homomorphic encryption configuration	Pass
TC-A06	test_mad __reputation_aggregation	Verify MAD+Reputation defensive aggregation	Pass
TC-A07	test_trust __update	Verify trust score update	Pass

Table 3: Model Aggregation Testing Results (7 tests passed, duration 0.032s)

Key Test Details:

- Correctly retained original parameters when no client updates were received.
- Single client aggregation returned identical parameters as the input.
- HE aggregation configuration verified with encryption ratio (0.02) and mask seed (42)
- Reputation scores calculated for each client in defensive aggregation round with precise round/trust score values

8.4 Key Test Results Details

8.4.1 FedAvg Aggregation Test (TC-A01)

- Input:
 - Client 1: All-ones 5x5 matrix
 - Client 2: All-twos 5x5 matrix
 - Client 3: All-threes 5x5 matrix
- Expected Result: All-twos 5x5 matrix
- Actual Result: All-twos 5x5 matrix

- Conclusion: FedAvg aggregation calculation is completely correct with matrix-level precision

8.4.2 Weighted Aggregation Test (TC-A04)

- Input:
 - Client 1: 50 samples, value = 1
 - Client 2: 100 samples, value = 2
 - Client 3: 150 samples, value = 3
- Weighted Avg = $\frac{50*1+100*2+150*3}{50+100+150} = 2.3333333333333335$
- Actual Result: 2.3333333333333333
- Conclusion: Weighted aggregation calculation is correct with negligible floating-point precision difference

8.4.3 MAD+Reputation Defensive Aggregation (TC-A06)

- Input:
 - Client 1: value = 1.0 (normal)
 - Client 2: value = 2.0 (normal)
 - Client 3: value = 10.0 (anomaly)
- Test Result:
 - • Defensive aggregation result: 1.6272842117423778
 - Normal average (excluding anomaly): 1.5
 - Full average (including anomaly): 4.333333333333333
- Round reputation scores: Client1 = 0.9447, Client2 = 1.0000, Client3 = 0.0263
- Round trust scores: Client1 = 0.9834, Client2 = 1.0000, Client3 = 0.7079
- Conclusion: Defensive aggregation effectively reduces the impact of anomalies, resulting in a close to normal average and far from the biased full average.

8.4.4 Trust Update Test (TC-A07)

- Anomalous client (Client3) trust score: Before Update=1.0000, After Update=0.8350
- Conclusion: Trust score of anomalous client is correctly reduced, dynamic trust update mechanism functions as expected

8.5 Test Summary

Test Module	Test Cases	Passed	Failed	Pass Rate	Duration
Model Type Testing	4	4	0	100%	0.005s
Flower Communication Protocol Testing	6	6	0	100%	0.417s
Model Aggregation Testing	7	7	0	100%	0.032s
Total	17	17	0	100%	0.454s

8.6 Test Execution Commands & Actual Execution

Execution Commands:

```
# Run all tests
```

```
python tests/run_all_tests.py
```

```
# Run individual tests
```

```
python tests/test_model_types.py
```

```
python tests/test_flower_protocol.py
```

```
python tests/test_model_aggregation.py
```

8.7 Test Conclusion

All 17 test cases passed with a 100% pass rate in the actual execution.

- Model Type Detection: Correctly identifies model structure differences and triggers warnings.
- Flower Communication Protocol: Handshake, heartbeat, parallel execution, timeout handling, and reconnection mechanisms all work properly.
- Model Aggregation: FedAvg aggregation, weighted aggregation, and defensive aggregation calculations are accurate; trust score update mechanism is effective.

The federated learning system meets all the tested functional requirements and demonstrates high reliability in core module operations, communication interactions and aggregation calculations.

9 Problems Encountered

9.1 Technical Challenges

9.1.1 Unreliable Communication Between Server and Clients

1. Problem Description

During the early rounds of testing, the communication between the Flower server and the client devices was not reliable. Some clients were unable to receive the latest updates of the weights, some rounds were terminated abruptly, and some updates were not sent to the server.

2. Reason Analysis

- **Hardware and Network Variability:** There were great variations in the processing capacities of the clients. The slow clients were the bottlenecks because all the clients were forced to wait for the slowest client.
- **Firewall blocking:** Some clients had their firewalls enabled, which blocked the ports allocated to the Flower protocol, leading to refused connections.

3. Remedial Action

We implemented a simple experiment where each client was only required to send updates of a single variable, (x_1) . This made the communication problems obvious. We also standardized the experiment environment because the firewalls were disabled and all the clients were connected through the same network.

9.1.2 Limited Training Visibility During Communication

1. Problem Description

During the process of the training round failing or a communication drop, it was not clear which client caused the error or whether the error occurred on the server or the client side. This is because there was no detailed runtime information.

2. Reason Analysis

The primary reason is that only console outputs were used, with no logs being recorded. This makes it difficult for post-mortem analysis. Moreover, all failures looked similar, with rounds simply stopping. This is true even if failures were caused by communication drops, invalid updates, or timeouts.

3. Remedial Action

A logging module is implemented that logs key events. This provides clear insights into where exactly the error occurred. This is beneficial for the debugging process.

9.1.3 Trade-off Between Security and System Performance

1. Problem Description

The implementation of homomorphic encryption caused increased computation time and communication costs. This is because homomorphic encryption is computationally expensive due to the larger size of the encrypted model parameters.

2. Reason Analysis

Paillier encryption is used in homomorphic encryption. Modular exponentiation is used in Paillier encryption. When all model parameters were encrypted, there were significant latency costs. This is because the encrypted parameters were larger than their plain-text versions.

3. Remedial Action

To resolve this problem, we decided to implement partial encryption. Only part of the model parameters is encrypted according to a certain ratio. This approach provides a good trade-off in terms of system performance and security. Experimental results have confirmed that using partial encryption is effective in reducing system cost while enhancing system security against gradient leakage attacks.

9.2 Teamwork Challenges

9.2.1 Language Barriers Among Team Members

1. Problem Description

During group meetings and team discussions, especially during in-depth team meetings, we noticed that some team members had difficulty following the team discussion. This caused some confusion among team members. As a result, there were some team members who were less active in team meetings.

2. Reason Analysis

The team is multicultural. Some team members were more comfortable communicating in English, while others were more comfortable communicating in Chinese. Using either language inevitably caused confusion among team members.

3. Remedial Action

To resolve this problem, we have implemented a basic mechanism where we ensure that we stop during in-depth team meetings to ensure that all team members understand what is being discussed. This mechanism has greatly reduced confusion among team members. Moreover, bilingual team members have been more active in helping team members understand complex team meetings. This change has greatly improved team meetings.

9.2.2 Role Ambiguity and Task Overlap

1. Problem Description

We didn't have a common agreement on who was handling what. Lacking clear ownership over different modules meant that people naturally gravitated toward familiar areas, resulting in redundant work and overall inefficiency.

2. Reason Analysis

Lack of role clarity caused overlapping responsibilities and repeated effort.

3. Remedial Action

We explicitly mapped out roles by assigning each person to particular system modules. To keep things coordinated, we now use ongoing group chats for quick updates and hold casual weekly meetings. This ongoing dialogue helps us stay in sync, monitor advancements, and catch any potential issues early on.

9.2.3 Uneven Work Distribution

1. Problem Description

There existed uneven work distribution in some stages of the project, with some members more actively engaged in the project's development and testing than others.

2. Reason Analysis

Such uneven distribution of work resulted from the different backgrounds of the members. Some members were more knowledgeable in Python and machine learning libraries, while others took more time to grasp the system architecture and tools.

3. Remedial Action

To address the problem of uneven work distribution, tasks were reallocated according to the capabilities and availability of each member. Simple documentation and UI tasks were given to those who needed more guidance, while more complex tasks such as aggregation and encryption were given to those who had more experience in these areas, making the project more efficient and ensuring its smooth progression.

10 Summary and reflection

10.1 Summary and Reflection on the Project

10.1.1 System Performance

In summary, the system has been performing well in the small-scale federated learning scenario. In the process of testing the system, the training process was able to converge in an acceptable number of rounds even when the data distributions of the clients were not identical.

The application of the custom aggregation methods such as weighted averaging and the MAD-based defense was effective in maintaining the stability of the model. Although the system is not optimized for the large-scale scenario, it performs well in the context of this project.

10.1.2 Communication Stability

Communication between the server and clients was found to be stable during the testing process. The Flower framework enabled the system to support communication with numerous clients and maintain synchronization, in addition to recovering from temporary disconnections.

All the communication features, including handshakes and heartbeat checks, performed as expected during the testing process. Although some minor delays were recorded in the system, the communication process performed without major hitches during the testing process.

10.1.3 Accuracy and Convergence Behavior

During the testing process, the system demonstrated accurate results and converged within a set number of training rounds in a controlled environment. However, when the system was subjected to real-world scenarios, the rate of convergence reduced significantly.

Although the system failed to converge as expected, it demonstrated consistency in results, indicating that the system is robust and reliable in its design and functionality.

10.1.4 Comparison with Expected Results

Most of the initial objectives of the project were met as expected. The system is capable of supporting communication and is modular and customizable in design and functionality. Moreover, the system is also capable of supporting partial homomorphic encryption, which is an added advantage in ensuring the privacy of clients and their data, although not entirely expected in the early stages of the project.

However, there are some limitations, particularly in scalability and optimization, which could be improved with more development time.

10.1.5 Planned vs Actual Progress

At the beginning of the project, the team had planned to implement the process in a particular order: communication, aggregation, and then testing. However, in reality, the process was not as linear as expected. It was more of an iterative process.

There were some components, such as communication and aggregation, which needed to be adjusted several times. This, of course, contributed to some delay, particularly in the development of complex features such as privacy and user interface improvements.

Although the development process was not strictly linear, the team was able to adjust accordingly to complete all the functionalities.

10.1.6 Team Collaboration

The experience of having a team with various backgrounds was both beneficial and challenging. On the one hand, having a team provided a wider scope of ideas. On the other hand, it was also challenging at times, particularly in the early stages of the development process.

At the beginning of the project, there was some confusion in the division of tasks, which contributed to some overlapping of work. However, towards the latter part of the development process, the team was able to work efficiently.

10.1.7 Learning and Reflection (LSEPI)

This project also provided valuable learning experience that went beyond the technical implementation. From a purely technical point of view, we also gained hands-on experience with federated learning, distributed systems, and basic privacy-preserving techniques.

This project also emphasized the importance of data privacy and responsible system design, especially in decentralized data systems.

Regarding professional skills, we have learned how to work in a team, how to communicate, and how to manage time. We have also learned that it is extremely important to be flexible when things do not go as planned.

Finally, playing around with our own aggregation techniques and security solutions also instilled in us a more exploratory mindset, which is essential in this field.

10.2 Time Plan Table

Work from October through December included dataset preparation, full Federated Learning pipeline development, experiments, interface refinement, and final documentation, completing all project objectives.

Table 4: Project Timeline and Task Completion

Time	Task	Status
17 Oct – 29 Oct	Research communication protocols; summarize FL/gradient descent concepts.	Done
29 Oct	Develop Flower-based & HTTP-based communication prototypes.	Done
5 Nov	Test aggregation workflow; begin interim report.	Done
12 Nov	Record demo video; continue report writing.	Done
19 Nov	Draft interim report complete.	Done
1 – 4 Dec	Finalize interim report; refine UI design.	Done
5 – 16 Dec	Select datasets; plan preprocessing pipeline.	Done
1 – 15 Jan 2026	Implement preprocessing.	Done
15 Jan – 9 Feb 2026	Full FL training; hyperparameter tuning.	Done
9 Feb – 2 Mar 2026	Benchmarking, heterogeneity experiments, and visual monitoring.	Done
2 – 16 Mar 2026	Implement advanced algorithms; robustness testing.	Done
16 – 23 Mar 2026	Finalize UI; optimize communication and system performance.	Done
23 Mar – 1 Apr 2026	Prepare final report, demo, and presentation.	Done

11 Conclusion And Future Work

11.1 Summary of Achievements

For this project, we developed a federated learning framework for image classification, with emphasis on communication, aggregation, and privacy. Federated learning enables multiple clients to jointly learn a common model without directly sharing their data, which enhances data privacy.

The main achievements in this project include the development of customized aggregation techniques. For instance, weighted averaging and MAD reputation filtering were used to mitigate the effects of abnormal client updates. These techniques significantly improved the stability of the model.

The other achievement in this project was the development of a privacy protection mechanism through the implementation of a partial homomorphic encryption scheme using the Paillier cryptosystem. Although it is not an encryption scheme, it is an additional security measure for the model parameters.

Finally, the developed federated learning framework was able to communicate efficiently using the Flower framework. It can support multiple clients, reconnect to the server, and learn continuously even in the event of client disconnection. From the test results, all components of the federated learning framework work as expected.

11.2 Limitations

Although the system performs well in the scope of the project, it is important to note that there are some limitations.

First of all, the system was tested with a small number of clients. Therefore, it is not clear how the system will perform in the real world. Furthermore, the partial encryption approach has some negative aspects. The problem with the full encryption approach is that it will greatly increase the computational time. Therefore, the full encryption approach was not used in this project.

Another limitation is that the project was tested with simple datasets such as MNIST. The datasets used in this project are simple and do not reflect the real world.

Lastly, the user interface is quite simple. The interface is functional, but it could be better.

11.3 Future Improvements

There are various ways this project can be improved in the future. One way the project can be improved is by making the system more scalable and supporting more clients. This will make the project more useful in real-world applications.

11.3.1 More Improvements:

- **Area of privacy:**

The project can be made more secure with the implementation of more advanced privacy features such as differential privacy and secure multi-party computation. This will make the project more robust and secure.

- **Complex Datasets and Models:**

Running it with complex datasets and models will give us a better idea of how the project works in the real world.

- **Area of user interface:**

The project can be made more user-friendly with the implementation of better visualization and monitoring features. This will make it easier for the user to get an idea of the mechanics behind the project.

- **Implementation of more robust fault-handling and aggregation features:**

This will make the project more robust and robust enough to be used in unpredictable conditions.

References

- [1] Educating Silicon (2024) *How much LLM training data is there, in the limit?* Educatingsilicon.com. Available at: <https://www.educatingsilicon.com/2024/05/09/how-much-llm-training-data-is-there-in-the-limit/> (Accessed: 5 November 2025).
- [2] Fang, M. *et al.* (2024) ‘Byzantine-Robust Decentralized Federated Learning’. Available at: <https://doi.org/10.48550/arxiv.2406.10416>.
- [3] Hain, M. (2025) *Exploring the Limits of Internet-Sourced Training Data*, Sapien.io, 27 October. Available at: <https://www.sapien.io/blog/exploring-the-limits-of-internet-sourced-training-data> (Accessed: 5 November 2025).
- [4] Kairouz, P. *et al.* (2019) ‘Advances and Open Problems in Federated Learning’. Available at: <https://doi.org/10.48550/arxiv.1912.04977>.
- [5] Meta (2024) *Introducing Meta Llama 3: The most capable openly available LLM to date*. ai.meta.com. Available at: <https://ai.meta.com/blog/meta-llama-3/> (Accessed: 5 November 2025).
- [6] Mothukuri, V., Parizi, R.M., Pouriyeh, S., Huang, Y., Dehghantanha, A. and Srivastava, G. (2021) A survey on security and privacy of federated learning. *Future Generation Computer Systems*, 115, pp.619–640.
- [7] Wu, W.-T. *et al.* (2025) ‘Horizontal Federated Learning: Research Status, System Applications and Open Challenges’, *Jisuanji Xuebao/Chinese Journal of Computers*, 48(1), pp.35–67. Available at: <http://cjc.ict.ac.cn/online/bfpub/wwt-2024118144807.pdf> (Accessed: 5 November 2025).
- [8] Xie, X., Hu, C., Ren, H. and Deng, J. (2024) A survey on vulnerability of federated learning: A learning algorithm perspective. *Neurocomputing*, 573, p.127225.
- [9] Geiping, J., Bauermeister, H., Dröge, H. and Moeller, M. (2020) Inverting gradients-how easy is it to break privacy in federated learning?. *Advances in Neural Information Processing Systems*, 33, pp.16937–16947.
- [10] Paillier, P. (1999) Public-key cryptosystems based on composite degree residuosity classes. In *International conference on the theory and applications of cryptographic techniques*, pp.223–238. Berlin, Heidelberg: Springer Berlin Heidelberg.
- [11] Zhu, L., Liu, Z. and Han, S. (2019) Deep leakage from gradients. *Advances in Neural Information Processing Systems*, 32.

- [12] European Union (2016) *General Data Protection Regulation (GDPR)*. Official Journal of the European Union.
- [13] He, C., Li, S., So, J., Zeng, X., Zhang, M., Wang, H., Wang, X., Vepakomma, P., Singh, A., Qiu, H. and Zhu, X. (2020) FedML: A research library and benchmark for federated machine learning. *arXiv preprint arXiv:2007.13518*.
- [14] Ingerman, A. and Shlens, J. (2020) *Federated learning: Collaborative machine learning without centralized training data*. Google Research.
- [15] Kairouz, P., McMahan, H.B., Avent, B., Bellet, A., Bennis, M., Bhagoji, A.N., Bonawitz, K., Charles, Z., Cormode, G., Cummings, R. and D'Oliveira, R.G. (2021) Advances and open problems in federated learning. *Foundations and Trends® in Machine Learning*, 14(1–2), pp.1–210.
- [16] Lim, W.Y.B., Luong, N.C., Hoang, D.T., Jiao, Y., Liang, Y.C., Yang, Q., Niyato, D. and Miao, C. (2020) Federated learning in mobile edge networks: A comprehensive survey. *IEEE Communications Surveys & Tutorials*, 22(3), pp.2031–2063.
- [17] Reina, G.A., Gruzdev, A., Foley, P., Perepelkina, O., Sharma, M., Davidyuk, I., Trushkin, I., Radionov, M., Mokrov, A., Agapov, D. and Martin, J. (2021) OpenFL: An open-source framework for Federated Learning. *arXiv preprint arXiv:2105.06413*.
- [18] Rieke, N., Hancox, J., Li, W., Milletari, F., Roth, H.R., Albarqouni, S., Bakas, S., Galtier, M.N., Landman, B.A., Maier-Hein, K. and Ourselin, S. (2020) The future of digital health with federated learning. *NPJ Digital Medicine*, 3(1), p.119.
- [19] Romanini, D., Davis, J., Georgiou, A., Holzer, J., Pan, J. and Van Essen, B. (2021) OpenFL: An open-source framework for federated learning. *arXiv preprint arXiv:2105.06413*.
- [20] Sheller, M.J., Edwards, B., Reina, G.A., Martin, J., Pati, S., Kotrotsou, A., Milchenko, M., Xu, W., Marcus, D., Colen, R.R. and Bakas, S. (2020) Federated learning in medicine: facilitating multi-institutional collaborations without sharing patient data. *Scientific Reports*, 10(1), p.12598.
- [21] Yang, Q., Liu, Y., Chen, T. and Tong, Y. (2019) Federated machine learning: Concept and applications. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 10(2), pp.1–19.
- [22] Krizhevsky, A., Sutskever, I. and Hinton, G.E. (2012) ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, 25. Available at: <https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.pdf>.

- [23] Jia, J., Liu, Z., Jia, A. and Feng, Y. (2026) *FedAlign: Defense against label flipping attacks in federated learning via dual-granularity distribution alignment*. Neurocomputing. Available at: <https://www.sciencedirect.com/science/article/pii/S0925231226006193> (Accessed: 26 Jan 2026).
- [24] Iglewicz, B. and Hoaglin, D.C. (1993) *How to Detect and Handle Outliers*. ASQC Basic References in Quality Control: Statistical Techniques.
- [25] Bonawitz, K., Ivanov, V., Kreuter, B., Marcedone, A., McMahan, H.B., Patel, S., Ramage, D., Segal, A. and Seth, K. (2017) *Practical secure aggregation for privacy-preserving machine learning*. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, pp. 1175–1191.

Appendices

A Meeting Minutes

A.1 Meeting Minutes 1 (17 October 2025)

Date: 17 October 2025

Time: 1:45 pm – 2:45 pm

Place: Library 2nd Floor Project Room 04

Chairman: Zhong Jun Khoo

Secretary: Aditya Agrawal

Attendees: Yanwen Mao, Aditya Agrawal, Zhong Jun Khoo, Mingyang Xue, Jiale Qin, Sirui Li, Xinchun Dai

Apologies: None

Agenda

- Introduction and overview of the project
- Explanation of key project concepts
- Clarification of deliverables and deadlines
- Group communication and weekly meetings
- Action items for next meeting

Discussion Points

- The tutor explained the goal: build an FL system for image classification with client-server communication.
- Key concepts clarified: Federated Learning and Gradient Descent.
- Deliverables and timeline were discussed.
- Communication setup requirements were clarified.
- Weekly meetings scheduled every Wednesday 9–10 AM.

Decisions Made

- Begin by understanding FL and setting up basic communication.
- Use simple datasets initially (e.g., MNIST).
- First milestone: establish basic architecture.

Action Points

Task	Responsible	Deadline
Draft meeting minutes	Aditya Agrawal	17 Oct 2025
Research communication protocols	All Members	22 Oct 2025
Summaries of FL and GD	All Members	21 Oct 2025
Prepare questions for next meeting	All Members	21 Oct 2025

A.2 Meeting Minutes 2 (29 October 2025)

Date: 29 October 2025

Time: 9:00 am – 10:00 am

Place: Library 2nd Floor Project Room 06

Chairman: Mingyang Xue

Secretary: Jiale Qin

Attendees: All members

Apologies: None

Agenda

- Review communication prototypes
- Discuss aggregation and next steps
- Plan interim report

Discussion Points

- Demonstrations of HTTP, gRPC, and Flower prototypes.
- Supervisor feedback on communication loops and customization challenges.
- MNIST chosen for testing.
- Plan for continuous bidirectional communication.

Decisions Made

- Flower chosen as main protocol.
- HTTP kept as backup.
- MNIST selected.

Action Points

Task	Responsible	Deadline
Draft meeting minutes	Jiale Qin	30 Oct 2025
Develop Flower communication	Aditya, Jiale, Sirui, Xinchun	5 Nov 2025
Develop HTTP communication	Khoo, Mingyang	5 Nov 2025
Progress report	All Members	5 Nov 2025

A.3 Meeting Minutes 3 (5 November 2025)

Date: 5 November 2025

Time: 9:00 am – 10:00 am

Place: Library 2nd Floor Project Room 06

Chairman: Sirui Li

Secretary: Xinchun Dai

Attendees: All members

Apologies: None

Agenda

- Review current prototypes
- Discuss aggregation and interim report
- Assign writing tasks

Discussion Points

- Review of Flask and Flower implementations.
- Aggregation and connection updates.
- Interim report discussion.
- Work division for next tasks.

Decisions Made

- Continue simple model aggregation testing.
- Begin writing interim report.

Action Points

Task	Responsible	Deadline
Draft minutes	Xinchun Dai	5 Nov 2025
Interim report sections	Aditya, Jiale, Sirui, Xinchun	12 Nov 2025
Testing aggregation	Khoo, Mingyang	12 Nov 2025

A.4 Meeting Minutes 4 (12 November 2025)

Date: 12 November 2025

Time: 9:00 am – 10:00 am

Place: Library 2nd Floor Project Room 06

Chairman: Zhong Jun Khoo

Secretary: Mingyang Xue

Attendees: All members (Except 1)

Apologies: Aditya Agrawal (Reason: Sickness)

Agenda

- Review convergence and demo progress
- Discuss report and video requirements
- Plan CNN integration

Discussion Points

- Convergence behavior of clients discussed.
- Advantages and challenges of FL summarised.
- Demo video required.
- Gradient extraction issue identified.
- Documentation progress reviewed.
- CNN architecture requirements discussed.

Decisions Made

- Record demonstration video this week.
- Continue refining prototype and report.

Action Points

Task	Responsible	Deadline
Record demo video	Khoo, Mingyang	19 Nov 2025
Continue report writing	All members	Ongoing

A.5 Meeting Minutes 5 (19 November 2025)

Date: 19 November 2025

Time: 9:00 am – 10:00 am

Place: Library 2nd Floor Project Room 06

Chairman: Jiale Qin

Secretary: Mingyang Xue

Attendees: All members

Apologies: None

Agenda

- Finalise interim report
- Plan UI and CNN phases
- Assign remaining sections

Discussion Points

- Code optimization
- Demo video recording plan
- Interim report improvements
- Preparation for CNN training before Christmas

Decisions Made

- Interim draft to be completed this week.
- Begin UI design and CNN planning immediately after submission.

Action Points

Task	Responsible	Deadline
Initial Design & UI	Khoo Zhong Jun	25 Nov 2025
Key Implementation Decisions	Sirui Li	25 Nov 2025
Prototype Results	Xue Mingyang	25 Nov 2025
Problems Encountered	Jiale Qin	25 Nov 2025
Time Plan & Front Page	Xinchen Dai	25 Nov 2025
LaTeX Formatting & Final Assembly	Aditya Agrawal	4 Dec 2025

A.6 Meeting Minutes 6 (3 December 2025)

Date: 3 December 2025

Time: 9:00 am – 10:00 am

Place: Library 2nd Floor Project Room 06

Chairman: Zhong Jun Khoo

Secretary: Aditya Agrawal

Supervisor: Yanwen Mao

Attendees: Sirui Li, Jiale Qin, Mingyang Xue, Xinchun Dai, Aditya Agrawal, Zhong Jun Khoo

Apologies: None

Agenda

- Review completed tasks
- Discuss interim report
- Work division and implementation planning
- Q&A session

Discussion Points

- **Review Completed Tasks**

The team reviewed the work completed in the previous week. UML diagrams including sequence diagrams, activity diagrams, and user diagrams were created to describe the initial system design and were included in the interim report.

The interim report was reorganized and simplified to meet the word limit requirement (below 5000 words). The structure was improved to ensure smoother transitions between sections.

- **Interim Report Discussion**

The supervisor confirmed that the overall structure of the interim report is appropriate. Minor adjustments to content were discussed and approved. No major issues were identified.

- **Work Division**

As the interim report submission deadline was the next day (4 December 2025), no further task allocation was required at this stage.

- **Second Half Project Direction**

The supervisor introduced six potential research directions for the second half of the project:

- Data homogeneity (data distribution differences across clients)
- Robustness (filtering malicious or low-quality gradients)
- Privacy (e.g., MPC or homomorphic encryption)
- Network structure (e.g., star topology)
- Contribution / free-rider detection
- Vertical federated learning

The team will select appropriate topics for further implementation in the next phase.

- **Q&A Session**

No questions were raised during this meeting.

Decisions Made

- Interim report confirmed and ready for submission.
- Second phase topics to be decided in the following week.

Action Points

Task	Responsible	Deadline
Decide second half project topic	All members	10 Dec 2025

A.7 Meeting Minutes 7 (10 December 2025)

Date: 10 December 2025

Time: 9:00 am – 10:00 am

Place: Library 2nd Floor Project Room 06

Chairman: Mingyang Xue

Secretary: Jiale Qin

Supervisor: Yanwen Mao

Attendees: Yanwen Mao, Aditya Agrawal (Team Leader), Zhong Jun Khoo, Mingyang Xue, Jiale Qin, Sirui Li, Xinchun Dai

Apologies: None

Agenda

- Meeting arrangement for upcoming stages
- Confirmation of selected research topics
- Implementation planning
- Privacy technique choices

Discussion Points

- **Meeting Arrangement for Upcoming Stages**

After discussion among all group members and the supervisor, the following schedule was agreed:

- No meetings during Study Week (10 Dec – 6 Jan)
- No meetings during Exam Week (7 Jan – 13 Jan)
- Online meetings only during Winter Vacation (13 Jan – 23 Feb)

- **Confirmation of Selected Research Topics**

Based on the six topics introduced in the previous meeting, the group decided to focus on **resiliency** and **privacy**.

The supervisor confirmed this decision and briefly explained their importance. The team was advised to review relevant literature and understand existing approaches before starting implementation.

- **Implementation Planning**

The supervisor suggested the following development steps:

- Start by building a basic federated learning system using the Flower framework

- Perform model aggregation and compare aggregated results with individual models, especially under heterogeneous data settings
- Gradually introduce resiliency and privacy mechanisms into the system

- **Privacy Technique Choices**

Three main privacy-preserving approaches were introduced:

- Multi-Party Computation (MPC)
- Homomorphic Encryption
- Differential Privacy

It was noted that implementing all three is not required. The group should select one method based on feasibility and team expertise. MPC was suggested as more practical from an engineering perspective, while the others require stronger mathematical background.

Decisions Made

- The group will focus on resiliency and privacy for the second phase.
- A basic federated learning system will be developed first.
- One privacy-preserving technique will be selected for implementation.

Action Points

Task	Responsible	Deadline
Draft meeting minutes and share with the team	Jiale Qin	10 Dec 2025
Research resiliency and privacy topics	All members	Before next online meeting
Build a basic federated learning system	All members	Before next online meeting
Implement resiliency feature	Two members	Before next online meeting
Implement privacy feature	Two members	Before next online meeting
Develop a simple front-end	Two members	Before next online meeting

A.8 Meeting Minutes 8 (21 January 2026)

Date: 21 January 2026

Time: 9:00 am – 10:00 am

Place: MS Teams

Chairman: Sirui Li

Secretary: Xinchun Dai

Supervisor: Yanwen Mao

Attendees: Yanwen Mao, Aditya Agrawal (Team Leader), Zhong Jun Khoo, Mingyang Xue, Jiale Qin, Sirui Li, Xinchun Dai

Apologies: None

Agenda

- Revision of completed work
- Planning for the upcoming week

Discussion Points

- **Review of Previous Work**

The team reviewed the tasks completed before the winter vacation, including initial progress on federated learning implementation and preliminary system design.

- **Task Division for Next Phase**

It was agreed that the next stage of development will be divided into three main parts:

- Privacy module
- Resiliency feature
- Simple front-end development

Each part will be assigned to different members to enable parallel development and improve efficiency.

Decisions Made

- The project will proceed with three parallel development tracks: privacy, resiliency, and front-end.
- Team members will be assigned to each track accordingly.

Action Points

Task	Responsible	Deadline
Implement resiliency feature	Two members	29 Jan 2026
Implement privacy feature	Two members	29 Jan 2026
Develop a simple front-end	Two members	29 Jan 2026

A.9 Meeting Minutes 9 (4 March 2026)

Date: 4 March 2026

Time: 11:10 am – 12:00 pm

Place: Library 2nd Floor Project Room 06

Chairman: Xinchun Dai

Secretary: Sirui Li

Supervisor: Yanwen Mao

Attendees: Yanwen Mao, Aditya Agrawal (Team Leader), Zhong Jun Khoo, Mingyang Xue, Jiale Qin, Sirui Li, Xinchun Dai

Apologies: None

Agenda

- Run federated learning on multiple devices and capture demonstration results
- Review completed tasks

Discussion Points

- **UI Design**

The current UI design was reviewed and considered satisfactory.

- **Robustness**

It was suggested to increase the number of training rounds to further evaluate system robustness.

- **Network**

Mingyang shifted focus to network topology, particularly studying the impact of client disconnections on model performance.

- **Privacy**

The current privacy-preserving mechanisms were evaluated and considered effective.

- **Implementation Planning**

Following the supervisor's suggestion, the UI will be enhanced with additional visualizations. These figures will illustrate:

- Robustness analysis
- Privacy performance
- Impact of client disconnections

Decisions Made

- Maintain the current UI design with additional visualization improvements.
- Focus on robustness, privacy, and network analysis for further experimentation.

Action Points

Task	Responsible	Deadline
Run FL on multiple devices and capture demo results	All members	Before next meeting
Enhance UI with visualizations	Assigned members	Before final demo
Increase training rounds for robustness testing	Assigned members	Before next meeting

A.10 Meeting Minutes 10 (18 March 2026)

Date: 18 March 2026

Time: 11:00 am – 12:00 pm

Place: Library 2nd Floor Project Room 06

Chairman: Mingyang Xue

Secretary: Jiale Qin

Supervisor: Yanwen Mao

Attendees: Yanwen Mao, Aditya Agrawal (Team Leader), Zhong Jun Khoo, Mingyang Xue, Jiale Qin, Sirui Li, Xinchun Dai

Apologies: None

Agenda

- Frontend demonstration of the federated learning system
- Discussion on training efficiency and user experience
- Improvement of demo design and storytelling
- Clarification of system modes
- Progress on attack and defense
- Task allocation and next steps

Discussion Points

- **Frontend System Demonstration**

The current frontend supports client-server mode switching, parameter configuration (e.g., number of clients and rounds), and dataset selection. The training process is partially visualized using logs and graphs. However, the system is relatively slow (approximately 2 minutes per round), which negatively affects the demonstration experience.

- **Training Efficiency and User Experience**

The supervisor highlighted that long waiting times reduce usability and may confuse users. Key issues include lack of progress indication and insufficient feedback during execution.

Suggested improvements include adding a “Stop Training and Show” (Skip) function, displaying results directly, reducing batch size, and optionally simulating training with pre-generated logs.

- **Demo Design and Storytelling**

It was emphasized that the demo should focus on storytelling rather than full system execution. Real-time training is not necessary. Instead, pre-generated results can be used to clearly demonstrate system behavior and performance improvements. The demo should be fast, clear, and visually convincing.

- **Visualization and Comparison**

The current system lacks clear comparison between different methods. It was suggested to include comparison plots showing baseline performance and improved results (e.g., with defense mechanisms). Visual outputs should clearly illustrate the benefits of the proposed approach.

- **System Modes and Usability**

Existing system modes are difficult for non-technical users to understand. Similar modes (e.g., random disconnect and availability) should be merged. Visual outputs should be used to explain the impact of different modes.

- **Privacy, Attack, and Defense**

The attack mechanism is based on information exchanged between client and server (e.g., gradients), without sharing raw data. However, the definition of attack success rate is unclear, and current visualizations are insufficient.

Improvements include clearly defining metrics and using visual comparisons (e.g., reconstruction results with and without privacy mechanisms).

- **Implementation Strategy**

To improve efficiency, it was recommended to avoid real-time training in the demo. Instead, use pre-trained results, stored data (e.g., JSON), and static images. User interactions should directly display results.

- **Workload Distribution**

To manage workload, the team will be divided into two groups: one focusing on report writing and the other on frontend/demo development.

- **Report and Evaluation Strategy**

The supervisor clarified that evaluation will focus on documentation quality and demo presentation rather than detailed code inspection. The report should clearly present methods, experiments, and results, while the demo should emphasize clarity and visual evidence.

Decisions Made

- Replace full training with a “Stop Training and Show” function.

- Use pre-generated results for all demo visualizations.
- Merge redundant system modes to simplify usability.
- Use static images instead of real-time computation for privacy modules.
- Focus on storytelling and visualization in the demo.

Action Points

Task	Responsible	Deadline
Implement “Skip” function	Assigned members	Before final demo
Prepare comparison plots	Assigned members	Before final demo
Merge system modes	Assigned members	Before final demo
Prepare privacy visualizations	Assigned members	Before final demo
Complete final report	All members	Submission dead- line
System testing	All members	Before submission

A.11 Meeting Minutes 11 (25 March 2026)

Date: 25 March 2026

Time: 5:00 pm – 6:00 pm

Place: Library 2nd Floor Project Room 06

Chairman: Sirui Li

Secretary: Xinchun Dai

Supervisor: Yanwen Mao

Attendees: Yanwen Mao, Aditya Agrawal (Team Leader), Zhong Jun Khoo, Mingyang Xue, Jiale Qin, Sirui Li, Xinchun Dai

Late: None

Apologies: None

Agenda

- Discuss the final project version
- Review the final report

Discussion Points

- **Final Report Review**

Each team member presented the section of the final report they were responsible for. The overall structure and content were reviewed to ensure consistency and completeness.

- **Final Project Version and System Testing**

The team reviewed the final version of the system and examined the results of system testing. It was confirmed that all core functionalities have been implemented and integrated successfully.

Decisions Made

- The team will be divided into two groups for the final stage:
 - One group will focus on poster preparation for the open day
 - The other group will focus on improving and finalizing the report

Action Points

Task	Responsible	Deadline
Edit and finalize the report document	Aditya	Before 1 Apr 2026
Prepare poster for open day	Zhong Jun Khoo	Before open day
Produce GRP promotional video	All members	Before 8 Apr 2026

User Manual

Federated Learning System Demo
for Server & Client

MINGYANG XUE (20617494) JIALE QIN (20616174)

Table of Contents

1. Prerequisites.....	2
2. Quick start.....	2
3. How to use core functions.....	2
4. Common questions and solutions.....	4
5. Glossary of terms.....	4

1. Prerequisites

- **Operating System:** Windows 10/11
- **Running Environment:** Python 3.6+

2. Quice start

2.1 Install dependencies

Run below code:

```
"pip install torch>=1.9.0 torchvision>=0.10.0 flwr>=1.0.0 numpy>=1.21.0  
matplotlib>=3.4.0 phe>=1.4.0"
```

2.2 Running the System

We will take the example of having multiple clients connected to a server on the same machine in parallel. Please run

```
"python app.py --port xxxx"
```

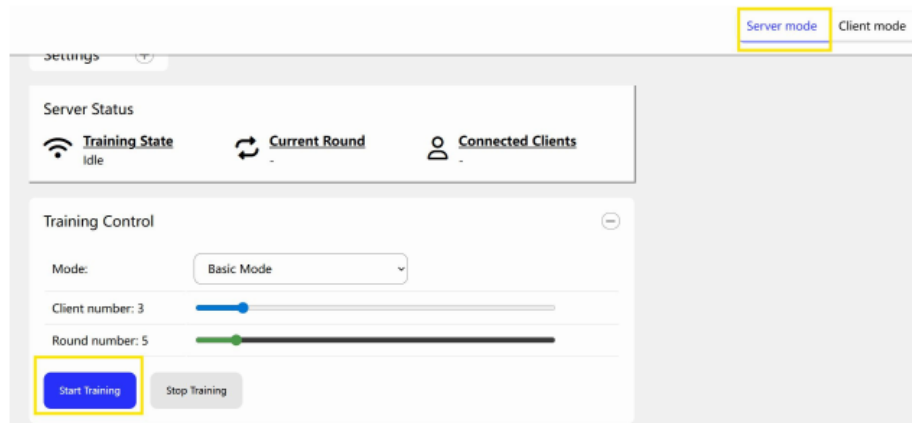
in different terminals. Each server and client needs to open an app. You can make the app run on different ports by modifying the port number. Please note that on the same device, avoid starting multiple apps on the same port.

2.3 Open the website

Open <http://localhost:xxxx> (change to your port number) in the browser

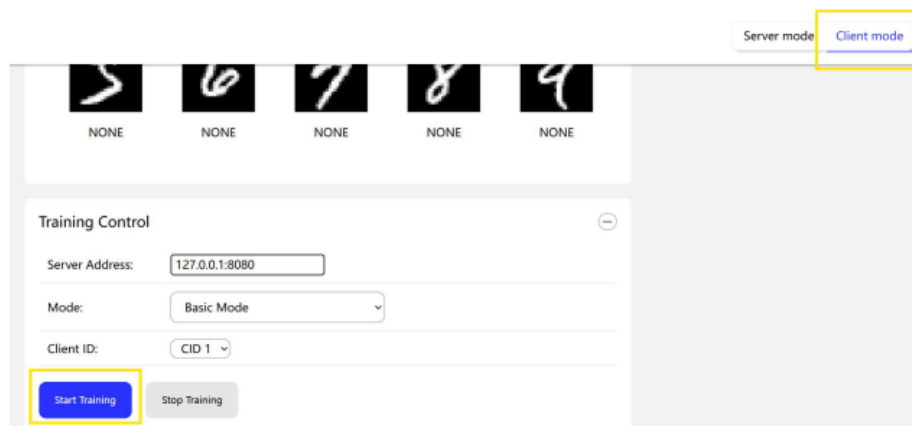
3. How to use core functions

- **Start FL Training (Server)**
 - In server page, click Start training
 - Expected result: Server shows training status and begins coordinating clients.



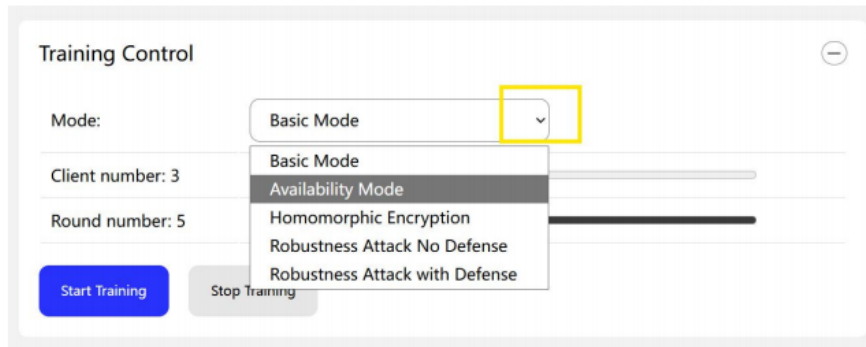
- **Start FL Training (Clients)**

- In client page, click Start training on each client page
- Expected result: clients connect to the server and training progress updates (UI/terminal).



- **Switch Mode**

- Click dropdown arrow next to 'Basic Mode' to switch to different modes.
- Expected result: the UI/feature behaviour changes according to the chosen mode.



4. Common questions and solutions

4.1 Clients disconnected

- Make sure the server has started.
- Check the server address is correct.
- Check the Internet connection.
- Check the firewall settings to ensure that the ports are not occupied.

4.2 Server crash

- Refresh the server and start training again.

5. Glossary of terms

Relevant parameters of the training:

'--num-clients' the number of clients

'--num-rounds' the number of training rounds

'--use-he' whether to enable homomorphic encryption

'--use-defense' whether use the defense mechanism (MAD + reputation system)

'--use-random-disconnection' whether use random disconnect to simulate.

'--use-history' whether use historical updates to mitigate the impact of disconnections

'--disconnect-prob' probability of server disconnection

'--max-stale-rounds' the longest duration of consecutive and intermittent states

'--he-encrypt-ratio' the proportion of homomorphic encryption
'--he-mask-seed' the seeds used in the homomorphic encryption
'--port' the port number of front end application
,

C Testing and Evaluation

C.1 Testing Overview

This project adopts a layered testing strategy to ensure the functional correctness of the federated learning system, the reliability of the communication protocol, and the accuracy of the aggregation algorithm. Testing covers three core modules: model type detection, Flower communication protocol, and model aggregation functionality.

C.1.1 Testing Strategy

Test Type	Test Object	Test Framework	Coverage Content
Unit Testing	Independent module functions	Python unittest	Model structure detection, parameter validation, aggregation calculation, trust score updates
Integration Testing	Client-server interaction	Flower + unittest	Handshake, heartbeat, parallel execution, timeout handling, reconnection
Defense Testing	Anomaly detection and aggregation	Custom + unittest	MAD+Reputation aggregation, outlier filtering, trust updates

C.1.2 Test Coverage

- Model Type Detection Module: Verifies that the system can identify different model structures and trigger warnings
- Flower Communication Protocol Module: Verifies the correctness and robustness of client-server communication sequences
- Model Aggregation Module: Verifies the mathematical correctness of FedAvg aggregation, weighted aggregation, and defensive aggregation

C.2 Test Cases and Results

C.2.1 Model Type Testing

ID	Test Case	Description	Status
TC-M01	test_model_structure_detection	Verify system can detect differences in model structures	Pass
TC-M02	test_server_parameter_comparison	Verify server can detect parameter count differences	Pass
TC-M03	test_warning_trigger_condition	Verify warning triggers when parameters mismatch	Pass
TC-M04	test_client_parameter_shape_logging	Verify server records parameter shapes for each client	Pass

Execution Result: 4 tests passed (0.005s)

Key Test Details:

- Successfully identified structural differences between standard and custom model layer configurations
- Accurately detected parameter count discrepancies (10 vs 8 parameters)
- Triggered precise warnings for parameter shape mismatch with client ID and detail information
- Correctly logged parameter shapes of multiple clients in standard format

C.2.2 Flower Communication Testing

ID	Test Case	Description	Status
TC-F01	test_client_handshake_sequence	Verify client handshake sequence correctness	Pass
TC-F02	test_heartbeat_protocol	Verify heartbeat communication works normally	Pass
TC-F03	test_parallel_client_execution	Verify multiple clients execute in parallel	Pass
TC-F04	test_round_completion_sequence	Verify complete training round sequence	Pass
TC-F05	test_client_timeout_detection	Verify timeout client detection	Pass
TC-F06	test_reconnection_handling	Verify disconnected client reconnection	Pass

Execution Result: 6 tests passed (0.417s)

Key Test Details:

- Handshake sequence strictly matched the expected connect → acknowledge → get_parameters → parameters
- Heartbeat protocol generated stable time-stamped heartbeat signals
- 4 clients achieved parallel execution with consistent start/end time stamps
- Complete training round sequence executed in standard order, aggregation triggered only after all client updates
- Accurately detected 1 timed-out client in test round
- Successfully recorded and handled client reconnection with missed round statistics

C.2.3 Model Aggregation Testing

ID	Test Case	Description	Status
TC-A01	test_fedavg_aggregation	Verify standard FedAvg aggregation	Pass
TC-A02	test_single_client_aggregation	Verify single client aggregation	Pass
TC-A03	test_empty_client_handling	Verify handling when no clients participate	Pass
TC-A04	test_weighted_aggregation	Verify weighted aggregation based on sample sizes	Pass
TC-A05	test_he_aggregation_structure	Verify homomorphic encryption configuration	Pass
TC-A06	test_mad_reputation_aggregation	Verify MAD+Reputation defensive aggregation	Pass
TC-A07	test_trust_update	Verify trust score update	Pass

Execution Result: 7 tests passed (0.032s)

Key Test Details:

- Correctly retained original parameters when no client updates were received
- Single client aggregation returned identical parameters as the input
- HE aggregation configuration verified with encryption ratio (0.02) and mask seed (42)
- Complete training round sequence executed in standard order, aggregation triggered only after all client updates
- Reputation scores calculated for each client in defensive aggregation round with precise round/trust score values

C.3 Detailed Result Analysis

C.3.1 FedAvg Aggregation Test (TC-A01)

Input Parameters:

- Client 1: All-ones matrix (5x5)
- Client 2: All-twos matrix (5x5)
- Client 3: All-threes matrix (5x5)

Expected Result: All-twos matrix (5x5)

Actual Result: All-twos matrix (5x5)

Conclusion: FedAvg aggregation calculation is completely correct with matrix-level precision.

C.3.2 Weighted Aggregation Test (TC-A04)

Input:

- Client 1: 50 samples, value = 1
- Client 2: 100 samples, value = 2
- Client 3: 150 samples, value = 3

Weighted Average Calculation:

$$\frac{50 \times 1 + 100 \times 2 + 150 \times 3}{50 + 100 + 150} = \frac{700}{300} = 2.333333333333335$$

Actual Result: 2.333333333333333

Conclusion: Weighted aggregation calculation is correct with negligible floating-point precision difference.

C.3.3 MAD + Reputation Defensive Aggregation Test (TC-A06)

Input:

- Client 1: value = 1.0 (normal)
- Client 2: value = 2.0 (normal)
- Client 3: value = 10.0 (anomaly)

Test Results:

- Defensive aggregation result: 1.6272842117423778
- Normal average (excluding anomaly): 1.5
- Full average (including anomaly): 4.333333333333333
- Round reputation scores: Client1 = 0.9447, Client2 = 1.0000, Client3 = 0.0263
- Round trust scores: Client1 = 0.9834, Client2 = 1.0000, Client3 = 0.7079

Conclusion: Defensive aggregation effectively reduces the impact of anomalies. The result is close to the normal average and significantly different from the biased full average.

C.3.4 Trust Update Test (TC-A07)

Anomalous client (Client 3) trust score change:

- Before update: 1.0000
- After update: 0.8350

Conclusion: Trust score of anomalous client is correctly reduced, and the dynamic trust update mechanism functions as expected.